

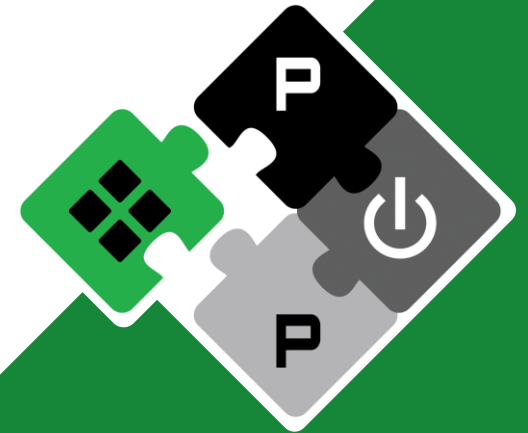
Semester Thesis presentation: Exploring Outer Product and Gustavson Dataflows for SpMM acceleration

Integrated Systems Laboratory (ETH Zürich)

Student:	Marco Ferroni	mferroni@ethz.ch
Supervisor:	Prof. Luca Benini	lbenini@iis.ee.ethz.ch
Tutors:	Simone Manoni	s.manoni@unibo.it
	Arpan S. Prasad	prasadar@iis.ee.ethz.ch
	Dr. Lorenzo Lamberti	llamberti@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!



pulp-platform.org

@pulp_platform

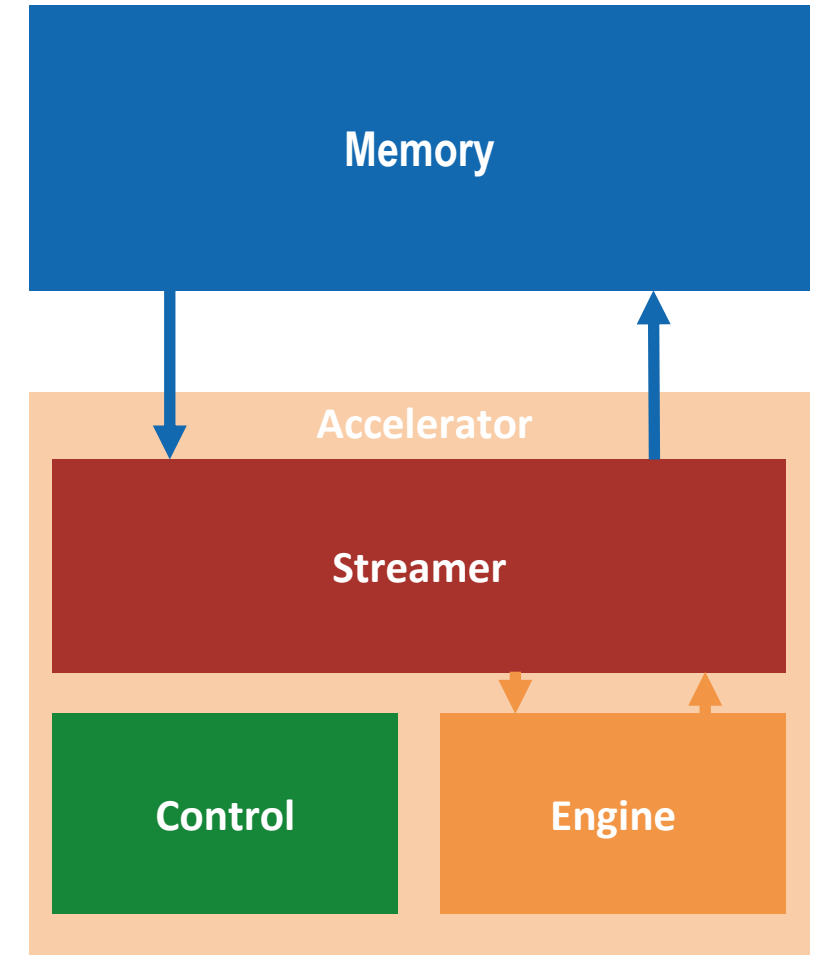
company/pulp-platform

youtube.com/pulp_platform



Motivation

- **General Matrix Multiplication (GEMM) is important**
 - In ML tasks activations (ReLU, etc.) and pruning introduce many zeros.
- **Taking sparsity into account has many benefits**
 - Avoid processing zeros $\Rightarrow$ less latency and memory footprint
- **Two possible options:**
 - Make the engine sparsity-aware
 - Make the streamer sparsity-aware
- **Chosen option:**
 - Make the streamer sparsity-aware and the engine agnostic



Accelerator's structure



Case study

- **Assumptions on data**

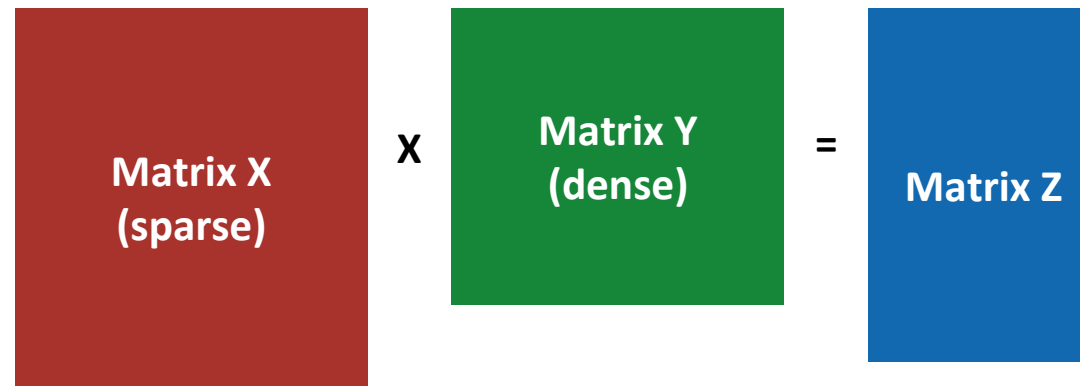
- X sparse
- Y dense

- **X encoded with bitmap format**

- The matrix is stored as its dense elements + a one-hot encoded mask of metadata

- **The engine takes chunks of the matrices**

- The size of the engine determines the size of the tiles of X and Y.



Naming conventions used



My contributions



- **Analytical evaluation**

- Compared Gustavson vs. Outer Product dataflows under various reuse strategies for Sparse Matrix Multiplication (SpMM)

- **Sparsity-aware extension of the Hardware Processing Engine's (HWPE) streamer**

- Integrated sparsity handling logic into the streamer
- Functionally verified on arbitrary matrix, varying sizes and sparsity levels

- **Design Evaluation**

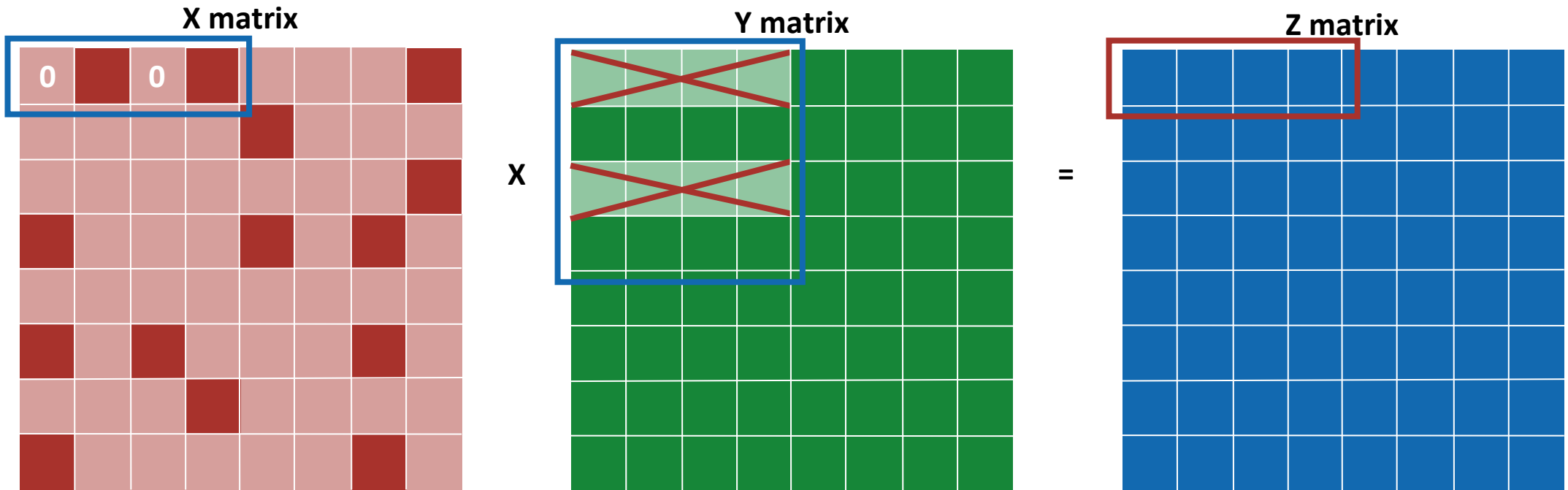
- Performance
- Area overhead



The idea

- **Avoid multiplications by zero**

- If the element in the row of X is a zero, I can skip the correspondent row of Y
- This way I avoid unnecessary loads and multiplications that consume bandwidth and cycles



Example with Gustavson

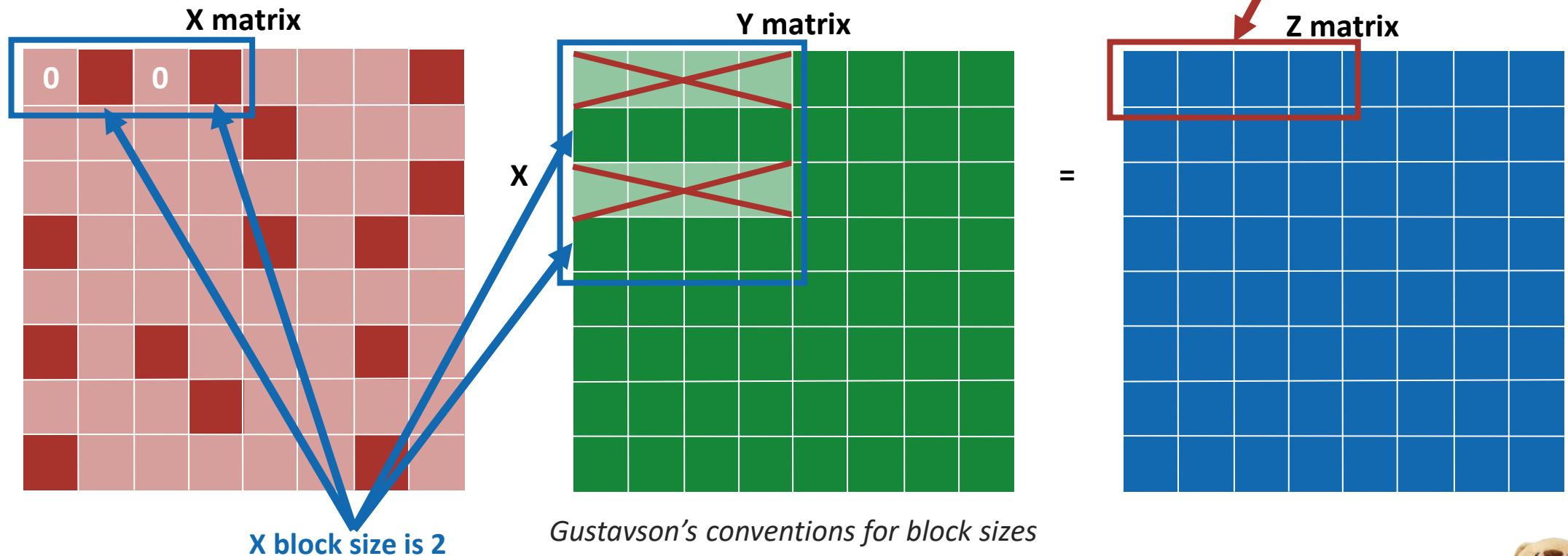


Dataflows options -Gustavson



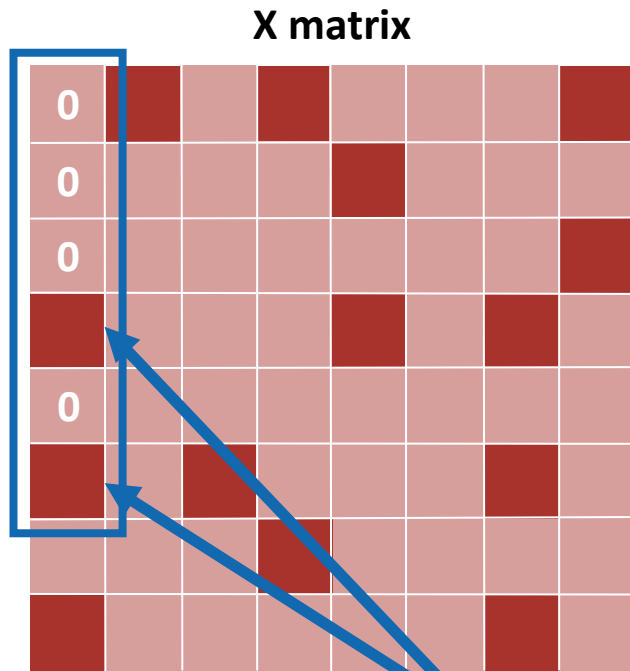
- **There are 3 possible options**

- X reuse: the X block is reused in the inner cycle
- Y reuse: the Y block is reused in the inner cycle
- Z reuse: the Z block is fully computed in the inner cycle

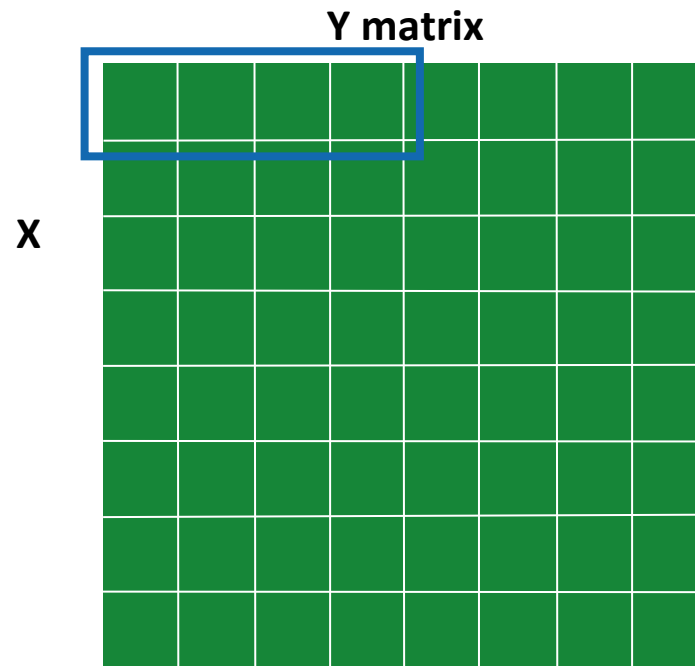


Dataflows options –Outer product

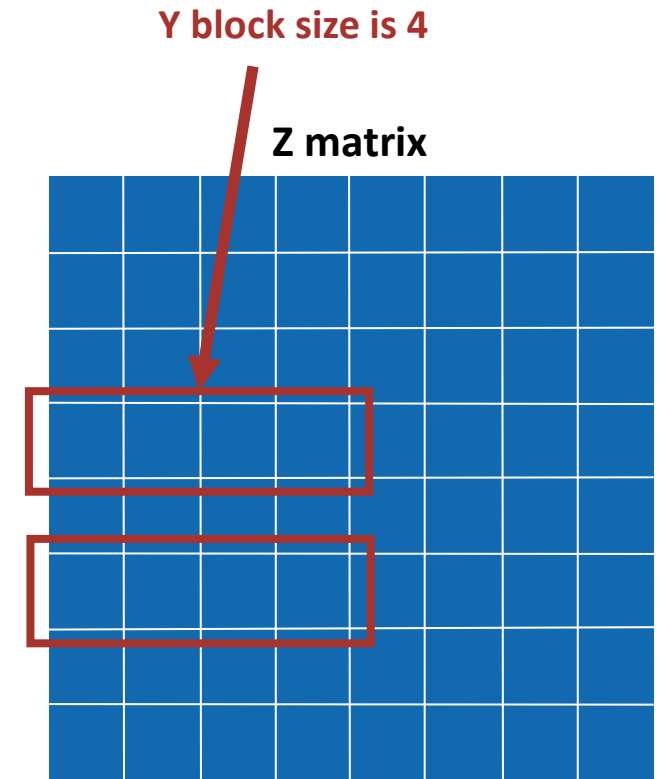
- There are 3 possible options
 - X reuse: the X block is reused in the inner cycle
 - Y reuse: the Y block is reused in the inner cycle
 - Z reuse: the Z block is fully computed in the inner cycle



X block size is 2



=



Outer product's conventions for block sizes

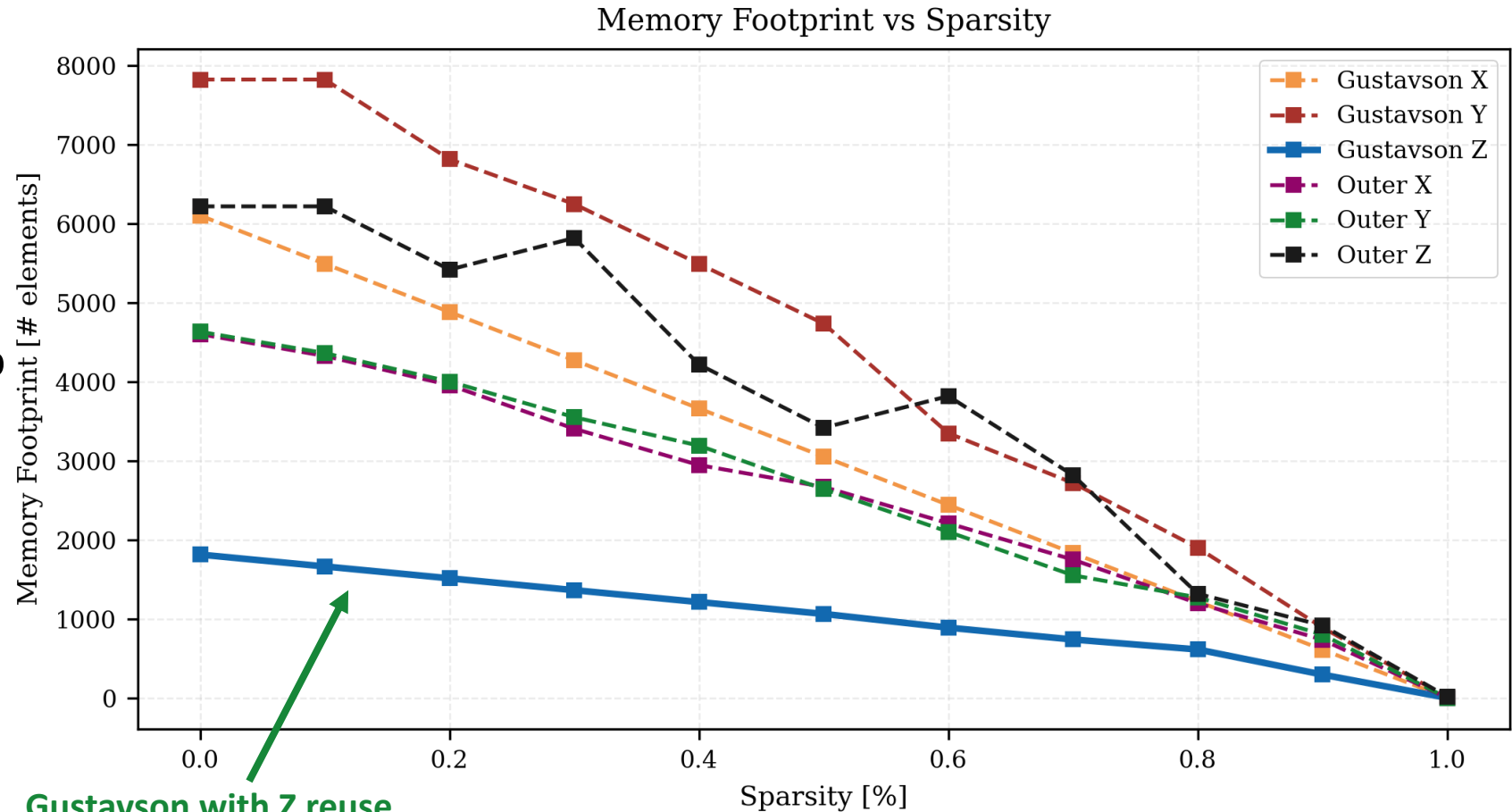


Analytical models comparison –Memory footprint



- **Assumptions:**

- MAC units = 4
- X rows = 32
- Y columns = 32
- Inner dim. = 32
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b



Gustavson with Z reuse
has the lowest memory footprint

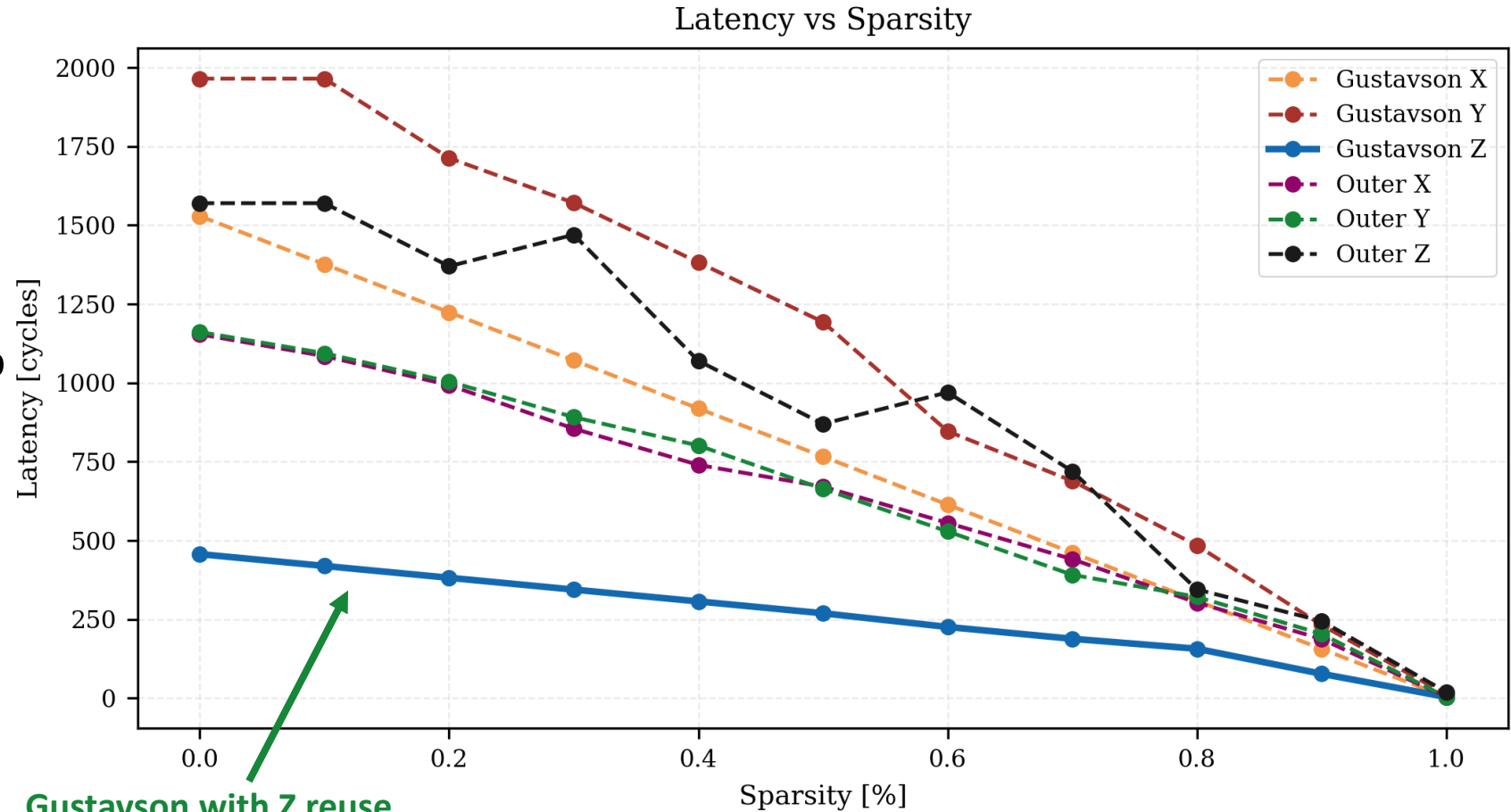


Analytical models comparison – Latency



- **Assumptions:**

- MAC units = 4
- X rows = 32
- Y columns = 32
- Inner dim. = 32
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b



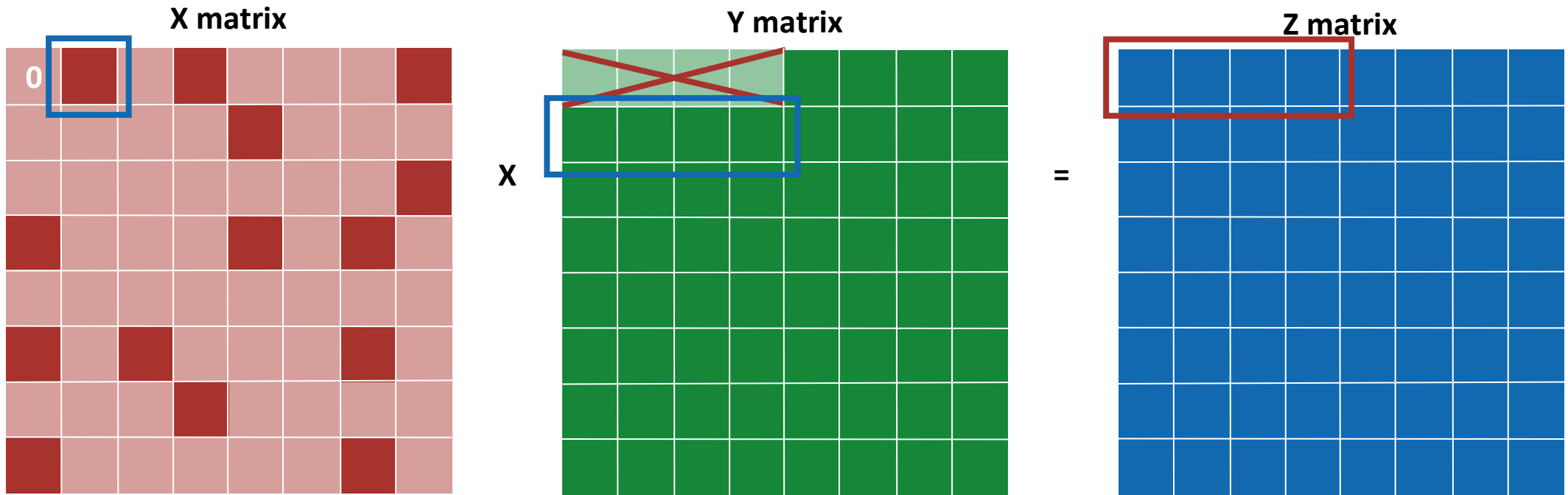
Gustavson with Z reuse
has the lowest latency



Final choice



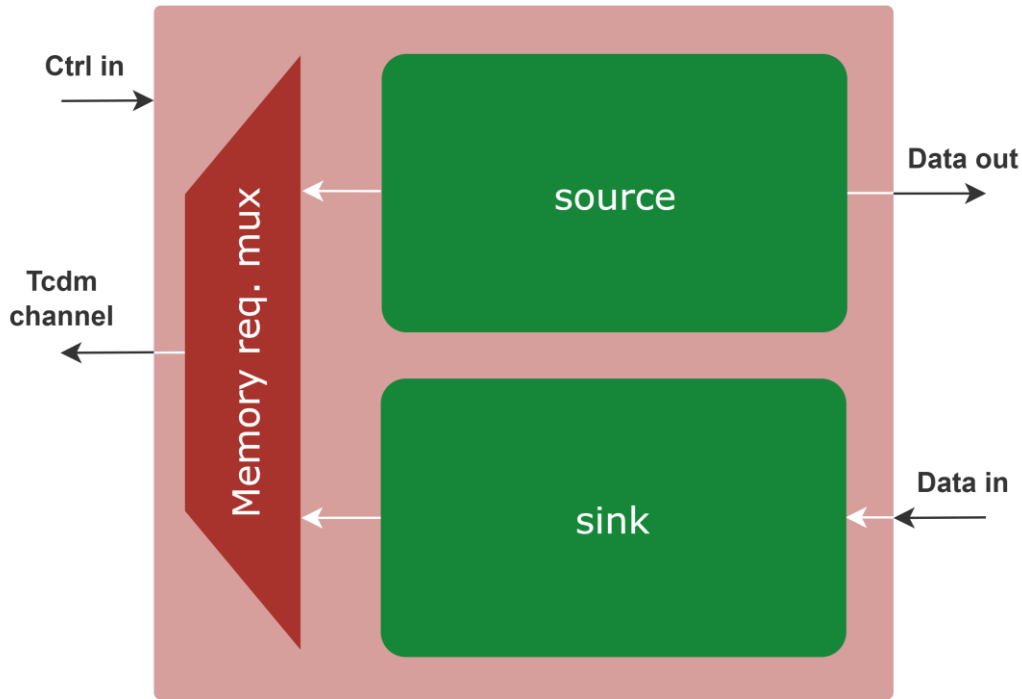
- Overall the Gustavson dataflow with Z reuse seems to be the best
 - Never loads chunks of Z from memory
 - Reuses the metadata as much as possible



Gustavson dataflow with Z stationary

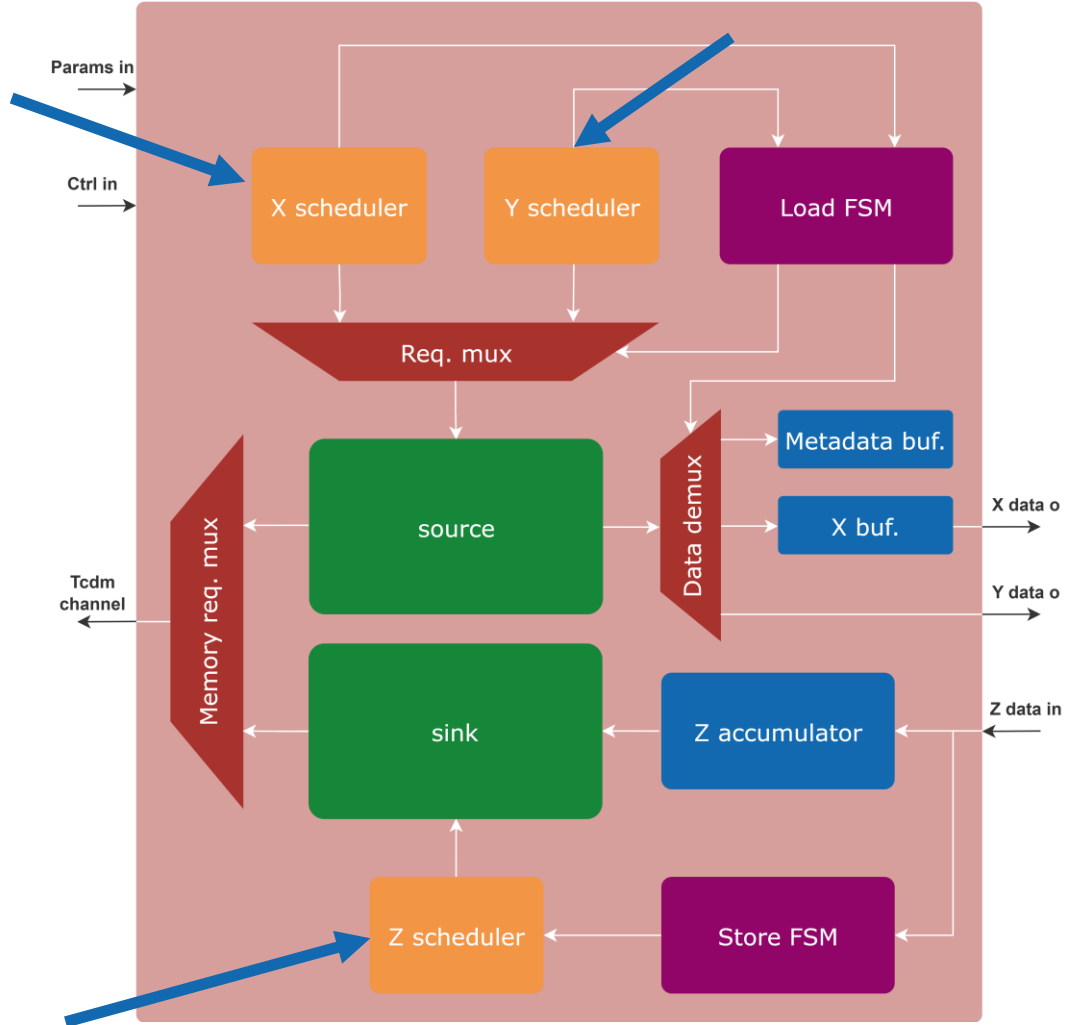


Accelerator's streamer architecture VS datamover one



Datamover's streamer architecture

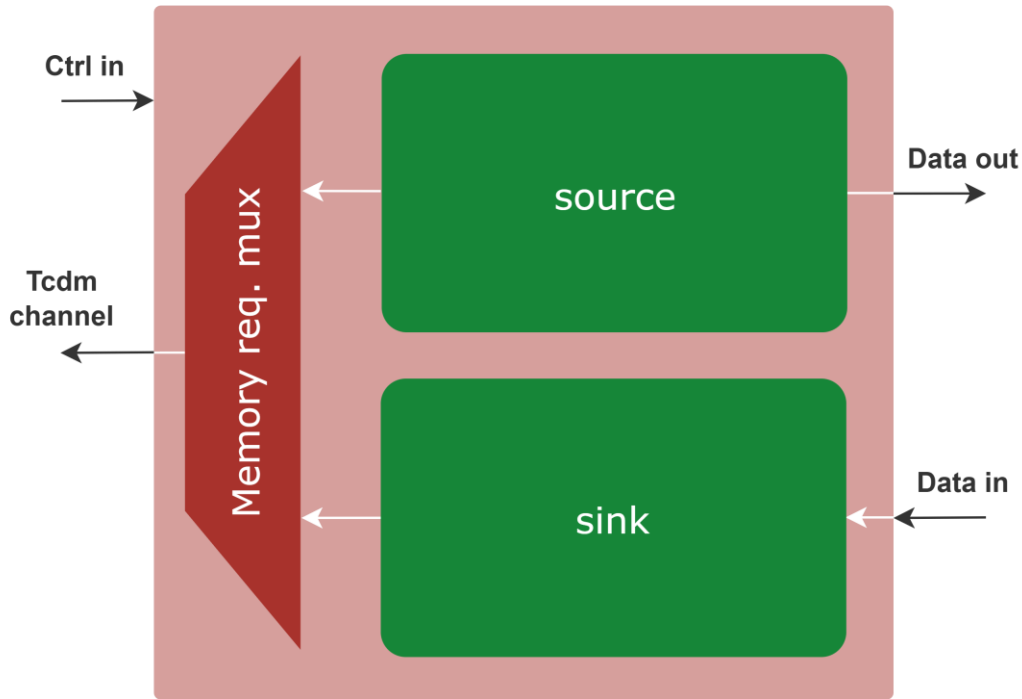
VS



Accelerator's streamer architecture

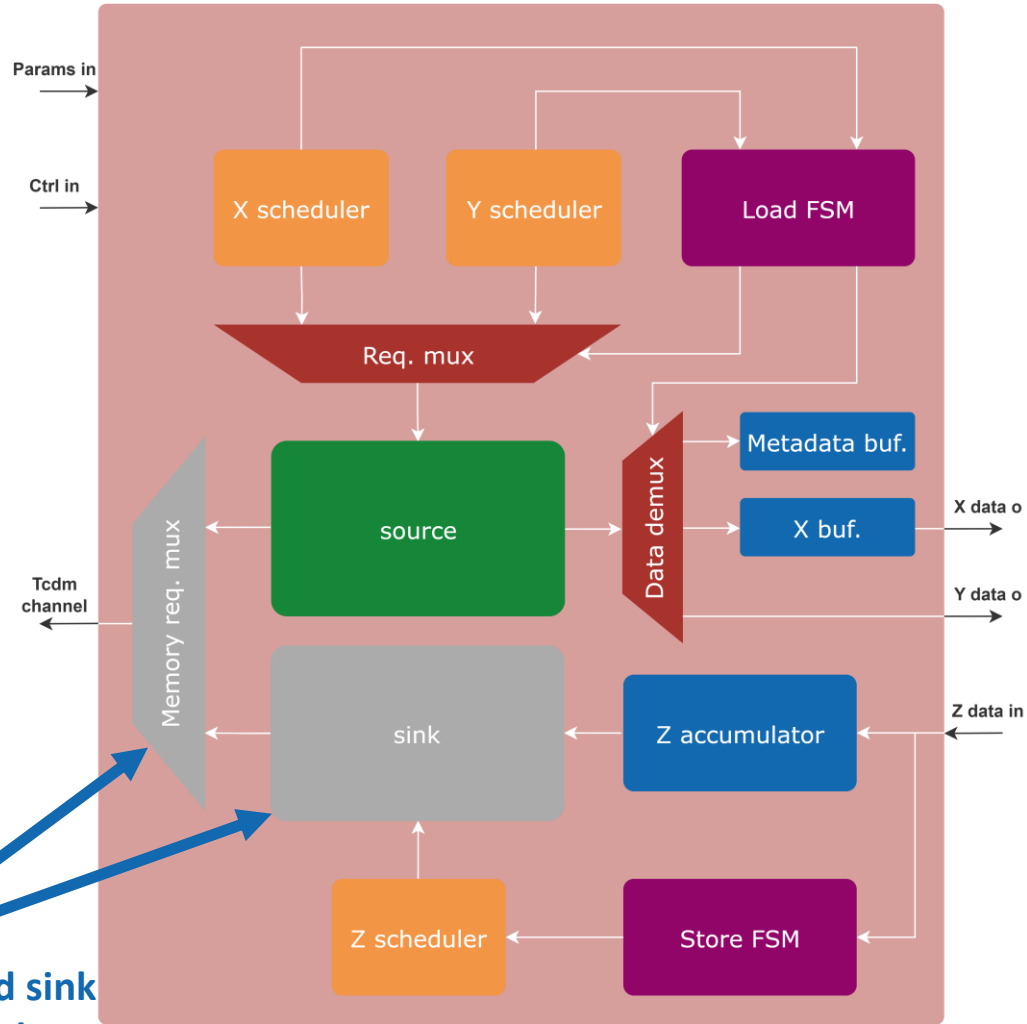


Accelerator's streamer architecture VS datamover one



Datamover's streamer architecture

VS

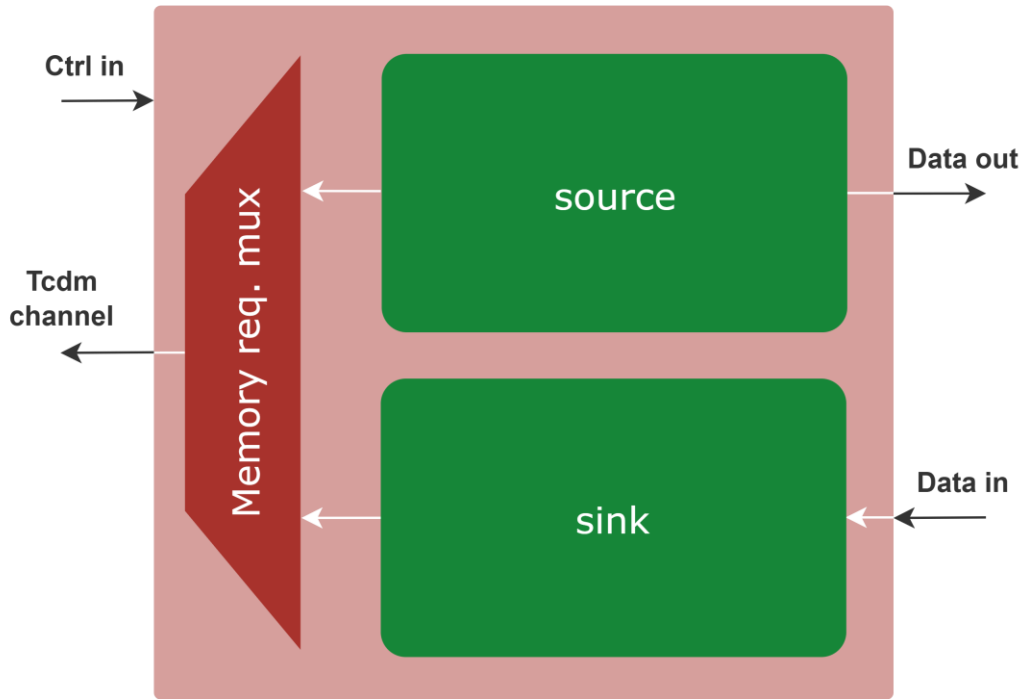


Accelerator's streamer architecture

The memory mux and sink were not modified

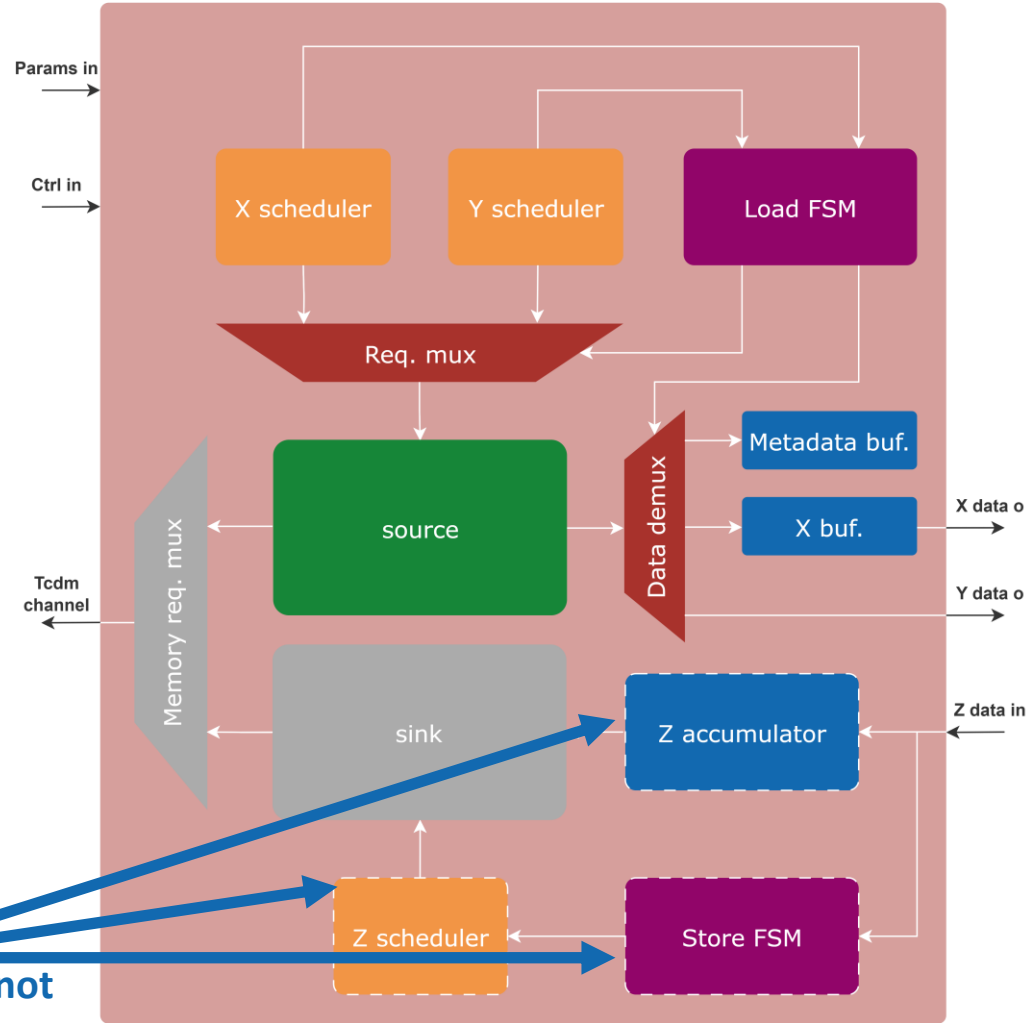


Accelerator's streamer architecture VS datamover one



Datamover's streamer architecture

VS

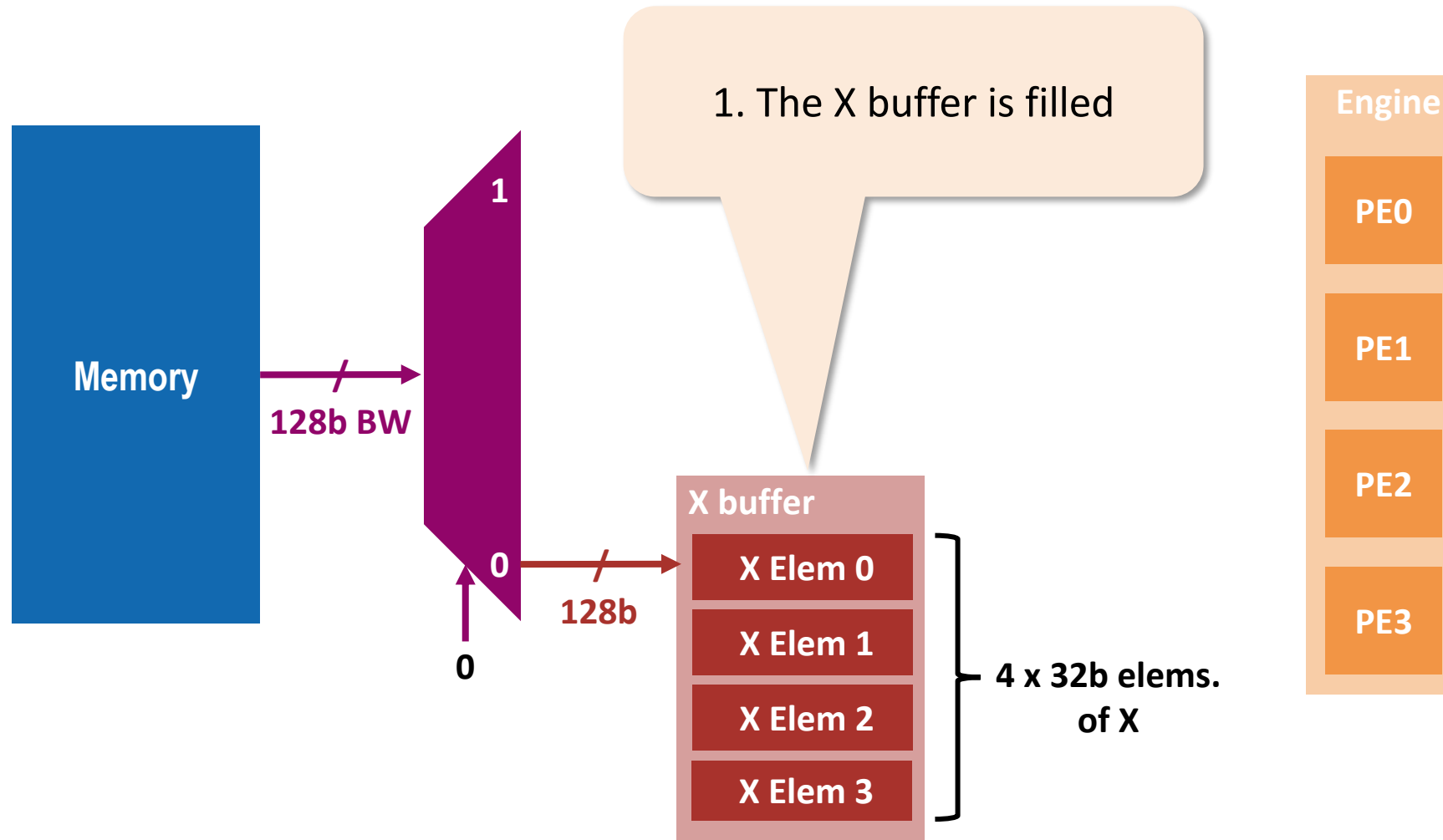


The storing part is not sparsity aware

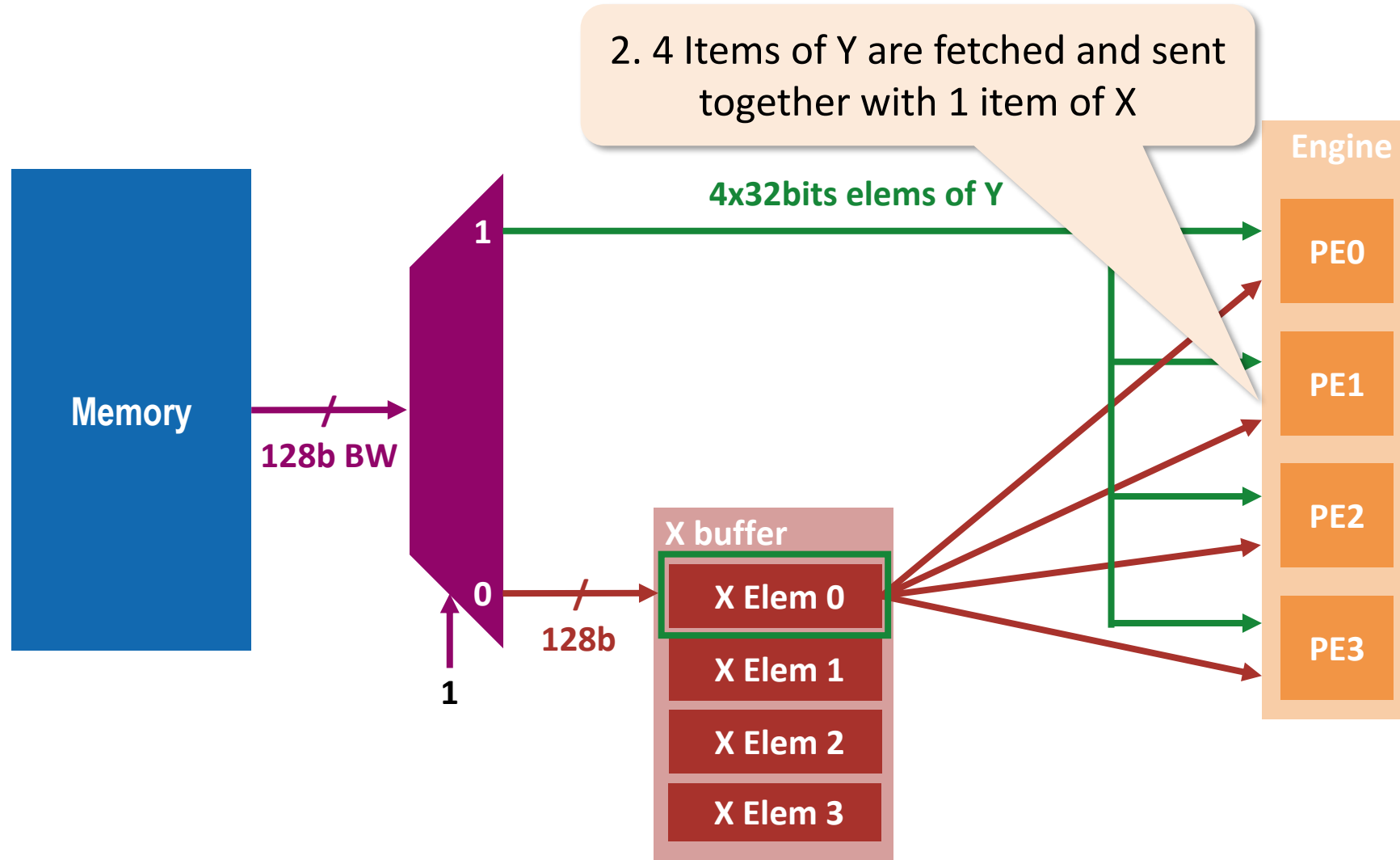
Accelerator's streamer architecture



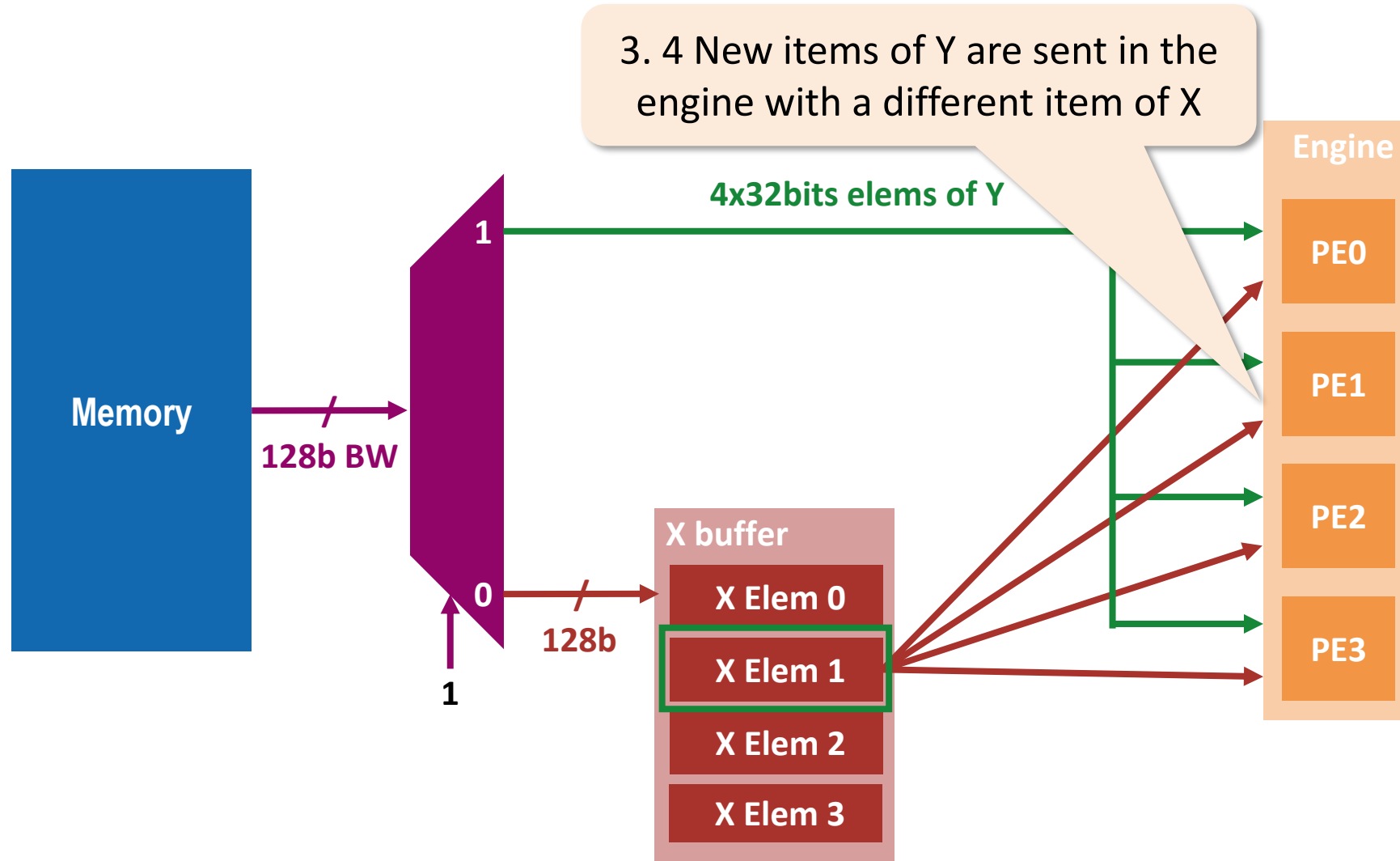
Streamer behavior –Loading part



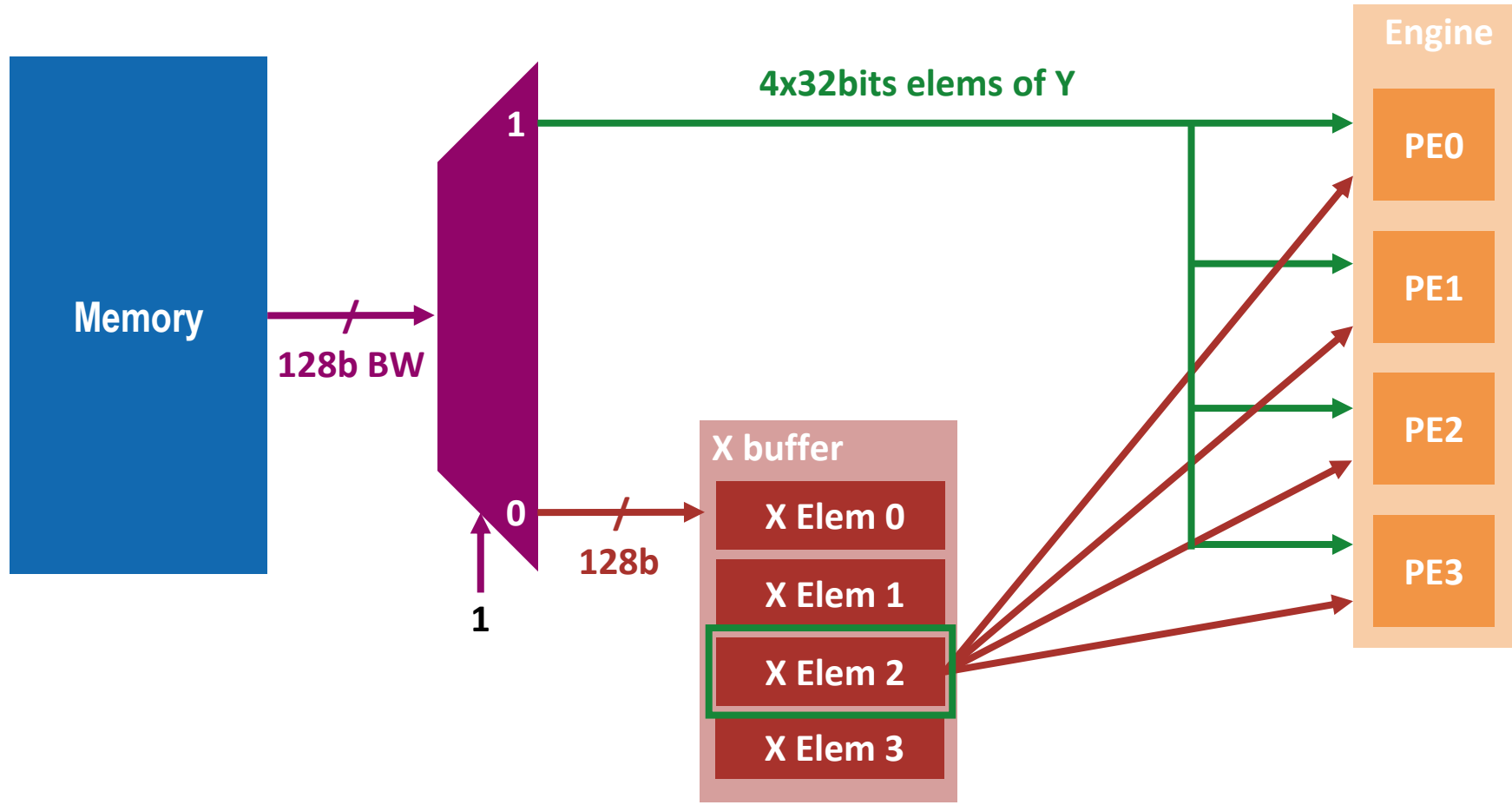
Streamer behavior –Loading part



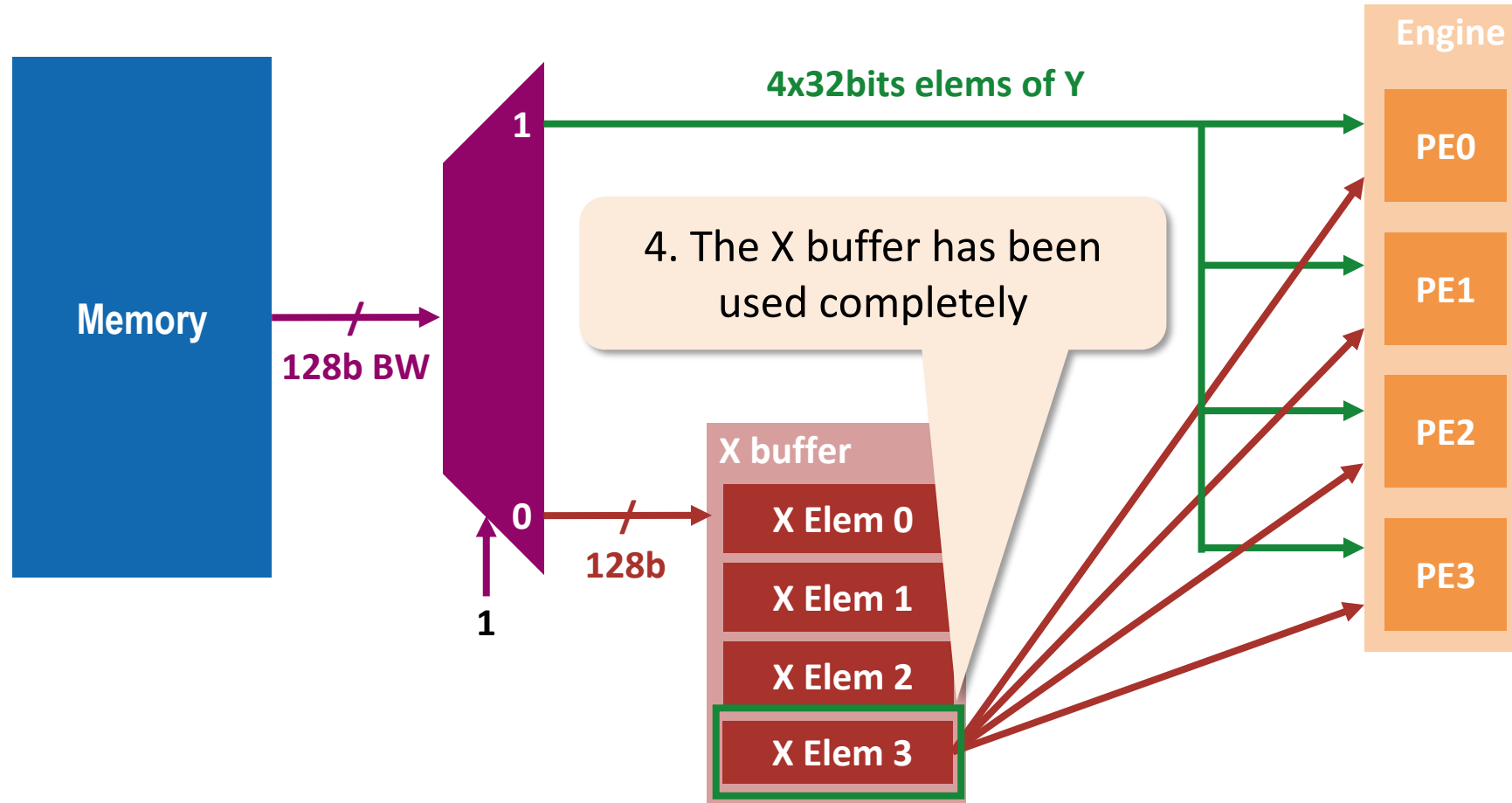
Streamer behavior –Loading part



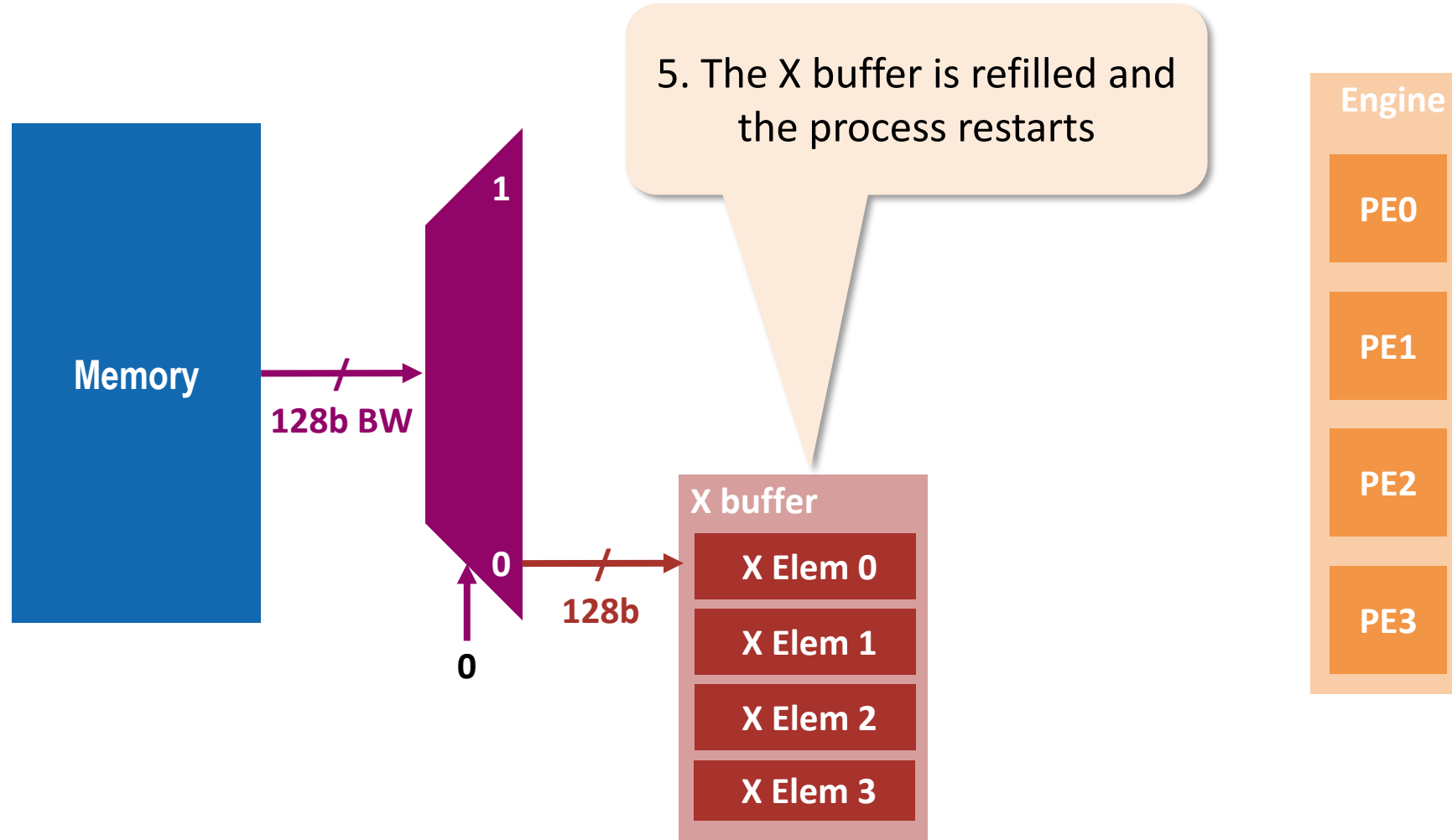
Streamer behavior –Loading part



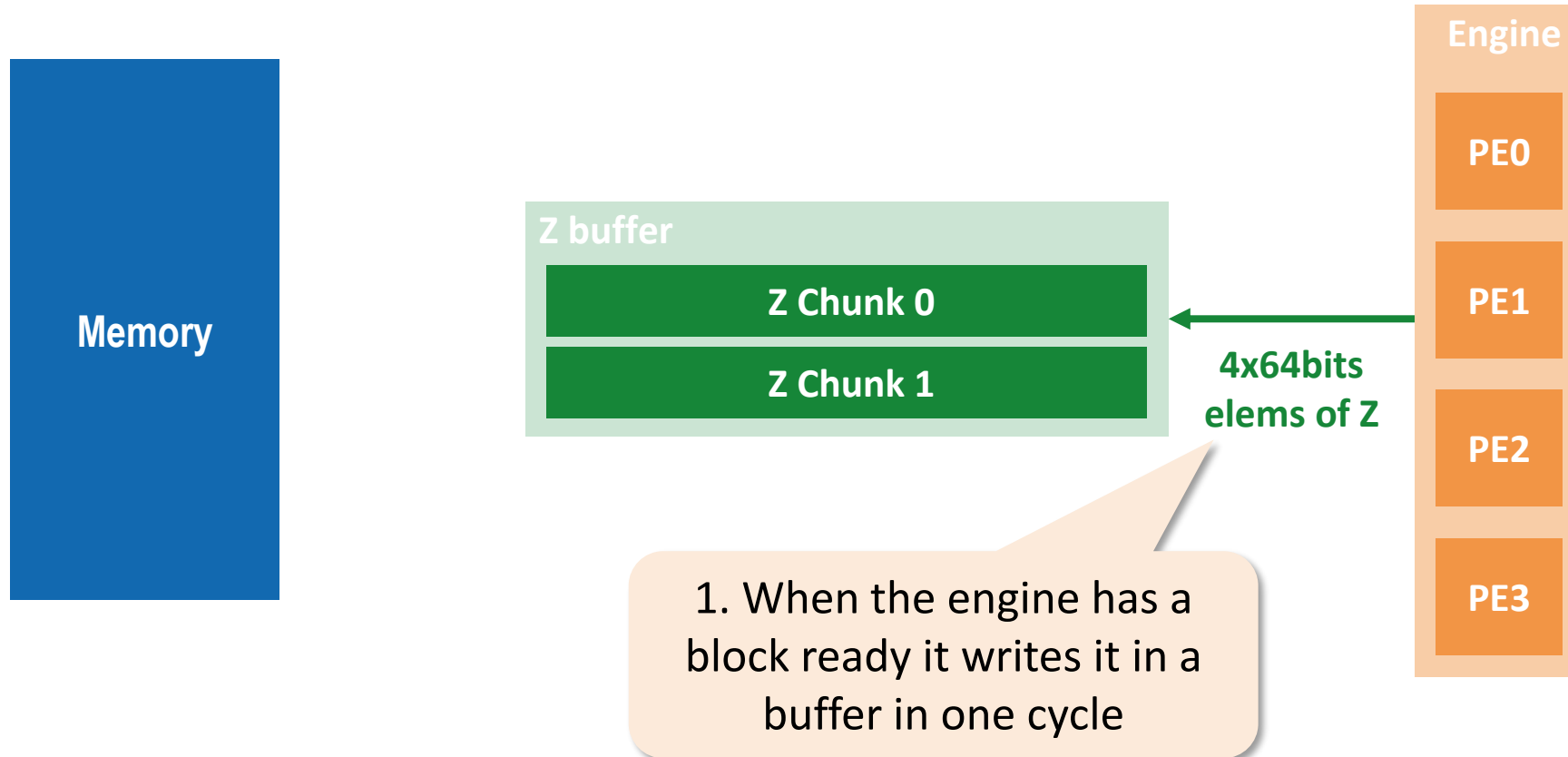
Streamer behavior –Loading part



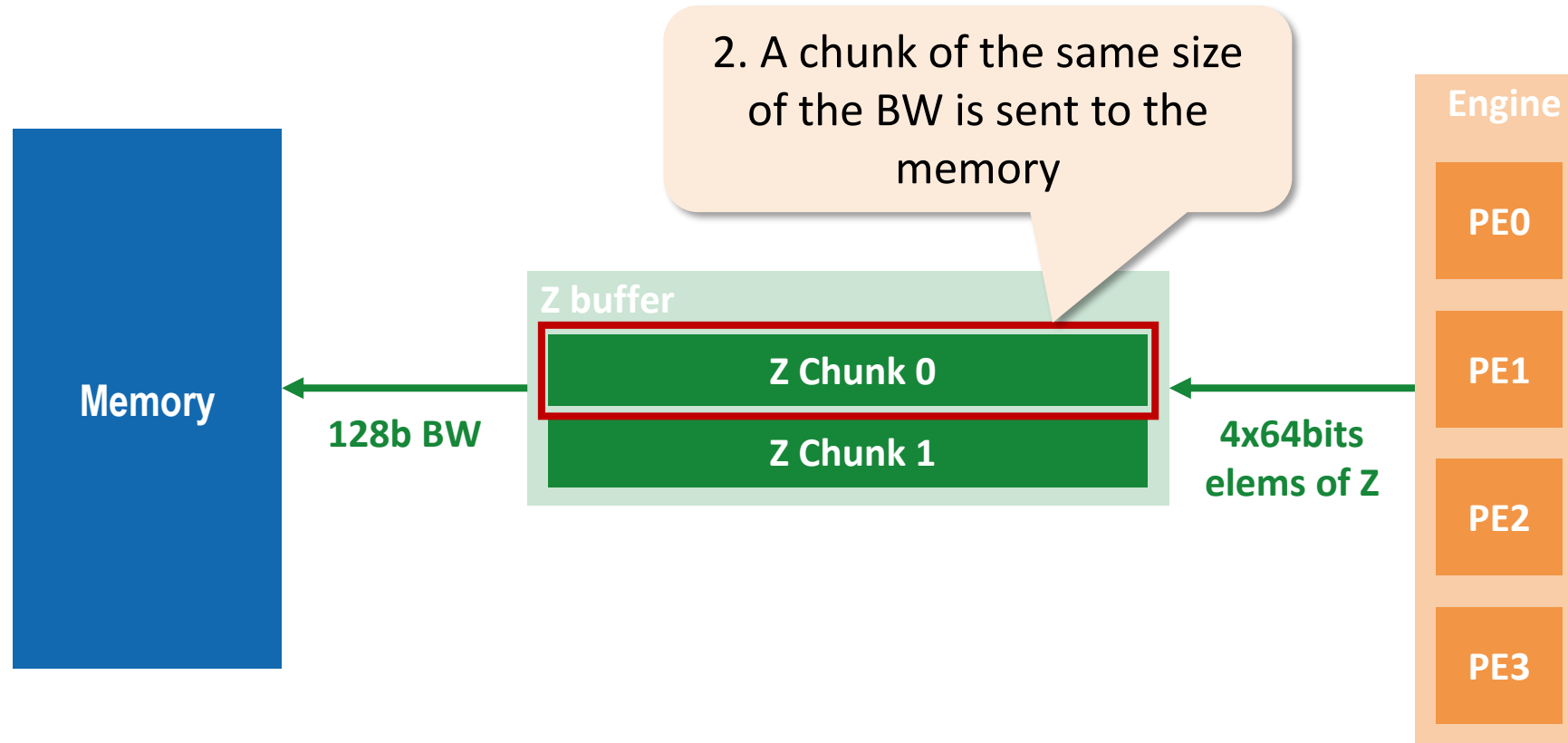
Streamer behavior –Loading part



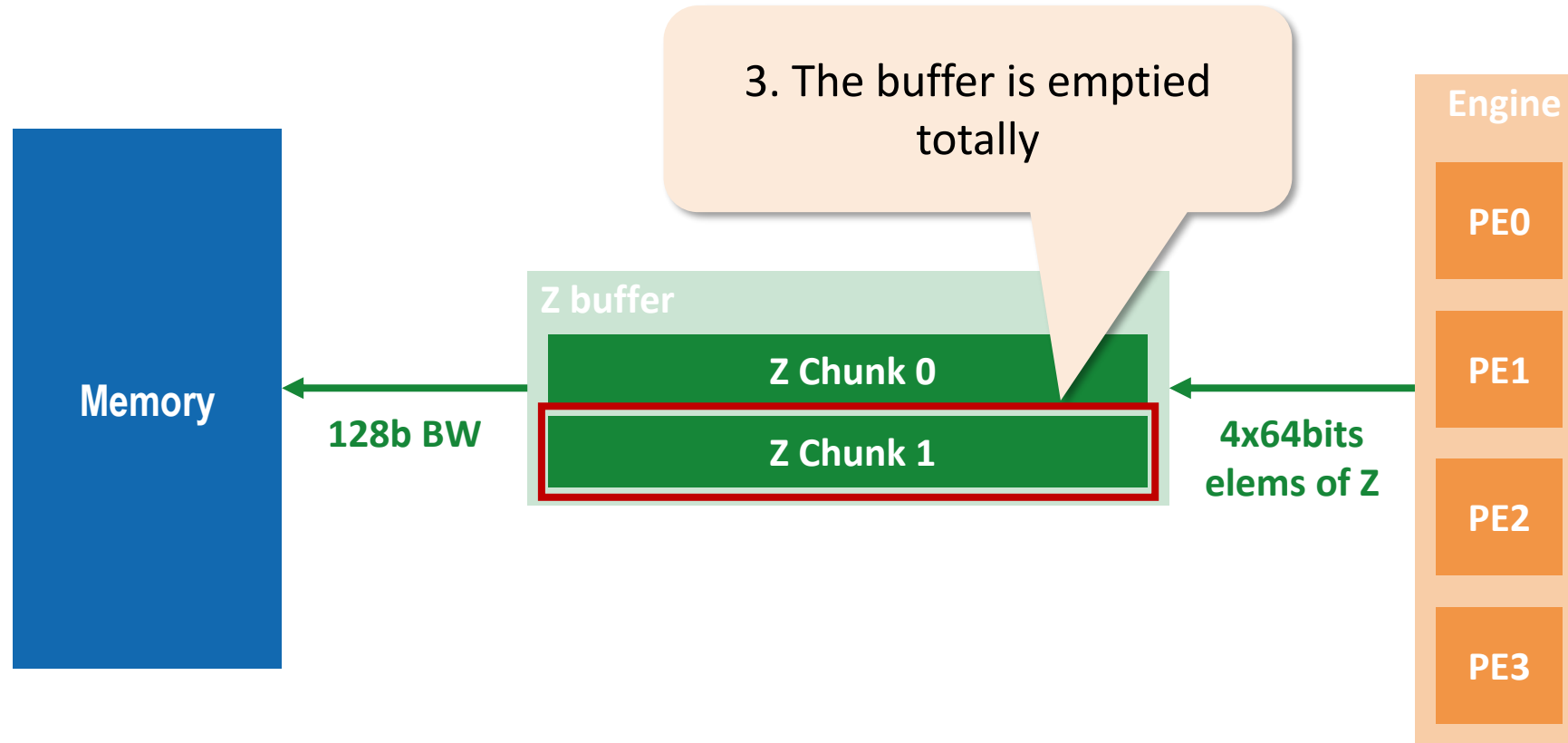
Streamer behavior –Storing part



Streamer behavior –Storing part



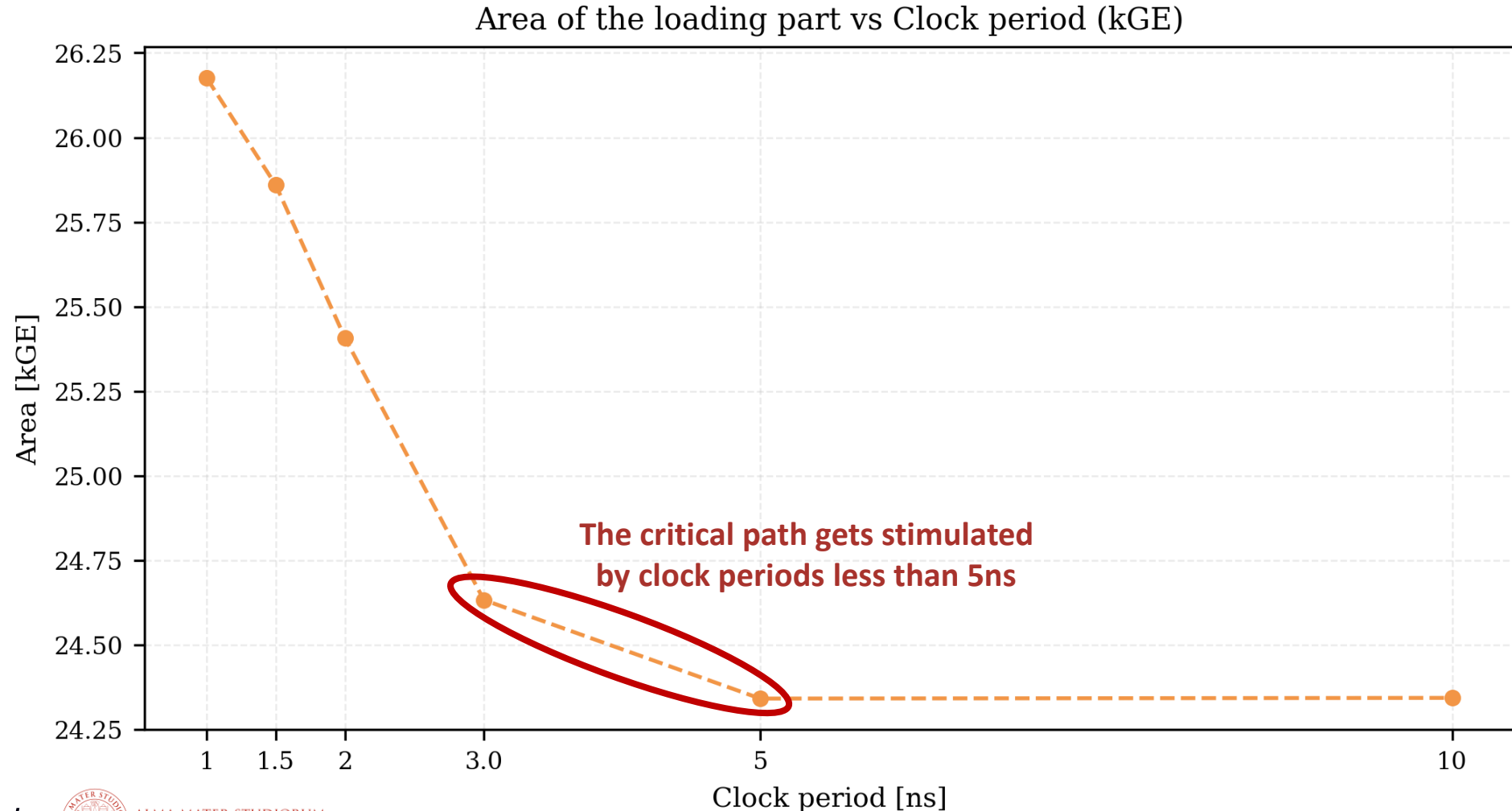
Streamer behavior –Storing part



Synthesis results- AT plot



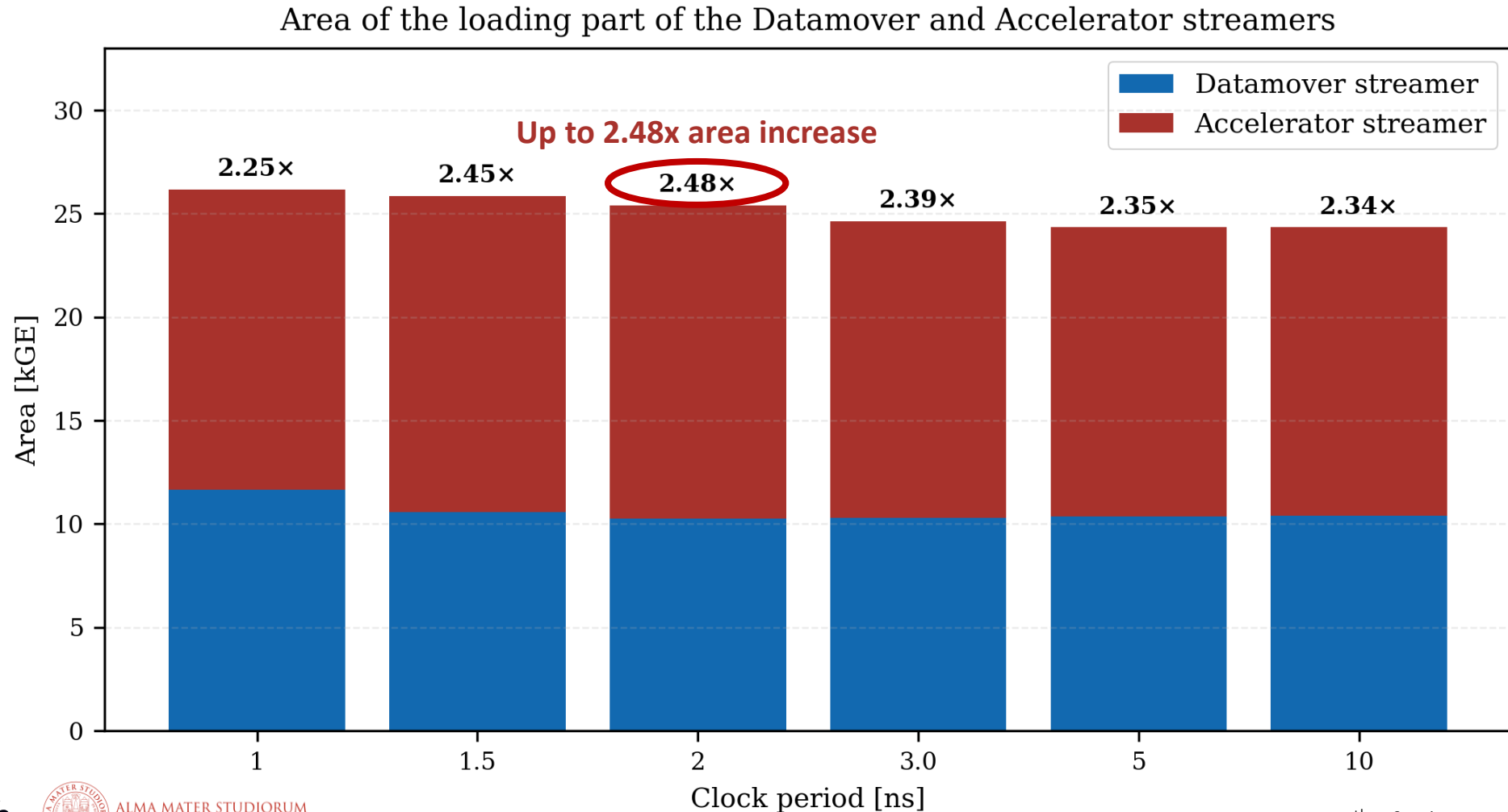
The plot reports the area considering only the loading part which was made sparsity-aware; synthesis was performed using the slow corner of gf22.



Synthesis results- Streamer VS Datamover



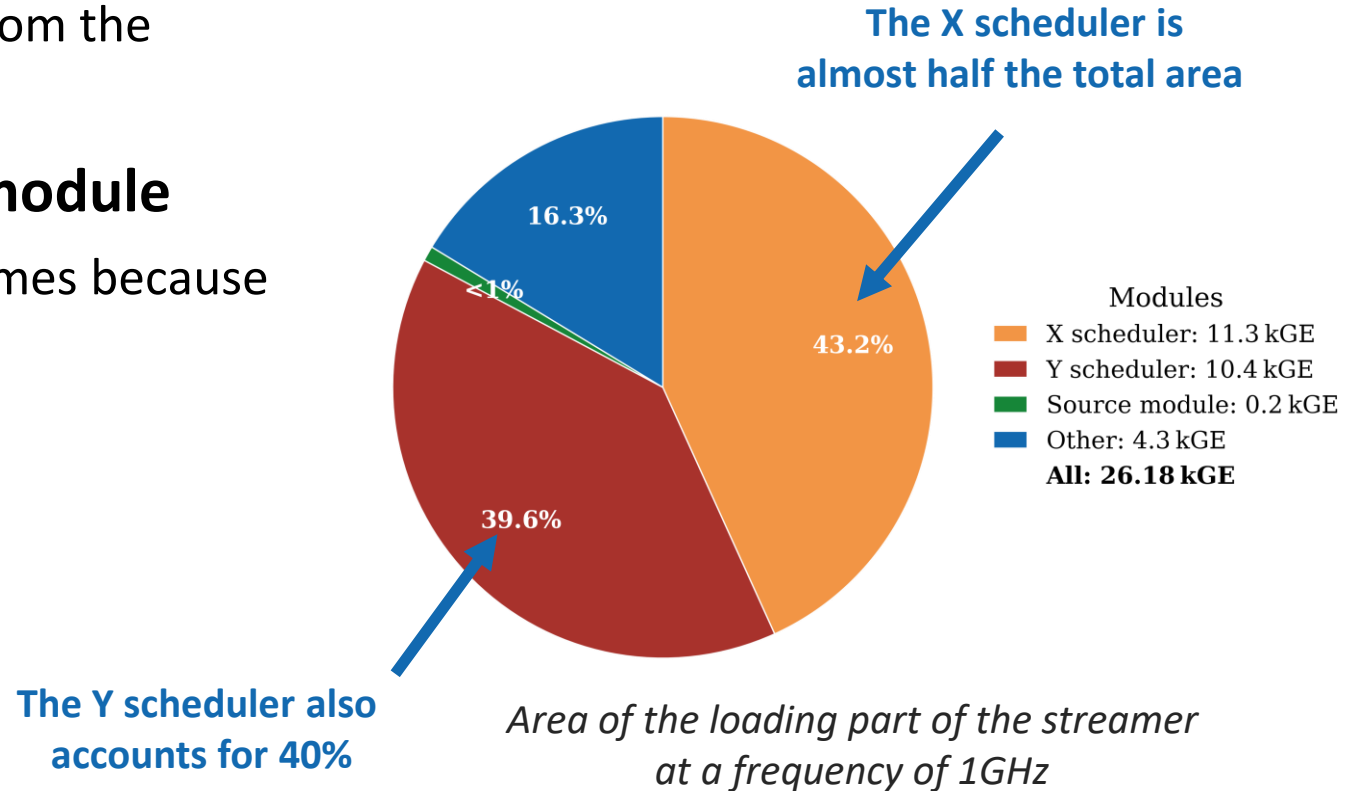
The plot reports the area difference only considering the loading part of the two designs, which was made sparsity-aware.



Synthesis results- Area contributions



- **Adding intelligence is expensive**
 - The biggest area contributions come from the schedulers
- **Different function of the source module**
 - The source was reduced of about 10 times because it's not "smart" anymore.

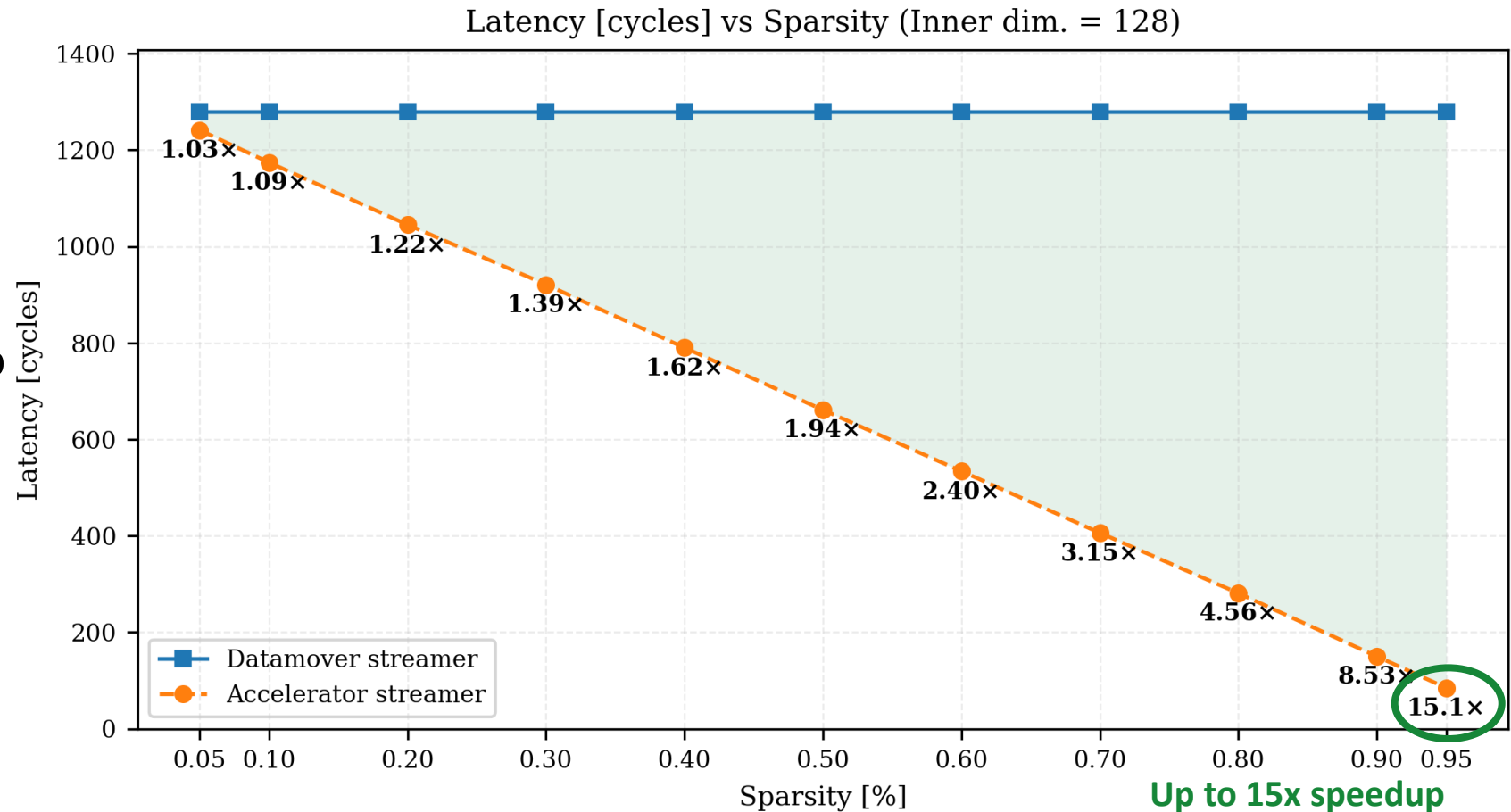


RTL results –Latency VS sparsity



- **Assumptions:**

- MAC units = 4
- X rows = 2
- Y columns = 16
- Inner dim. = 128
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b



Up to 15x speedup
with 95% sparsity

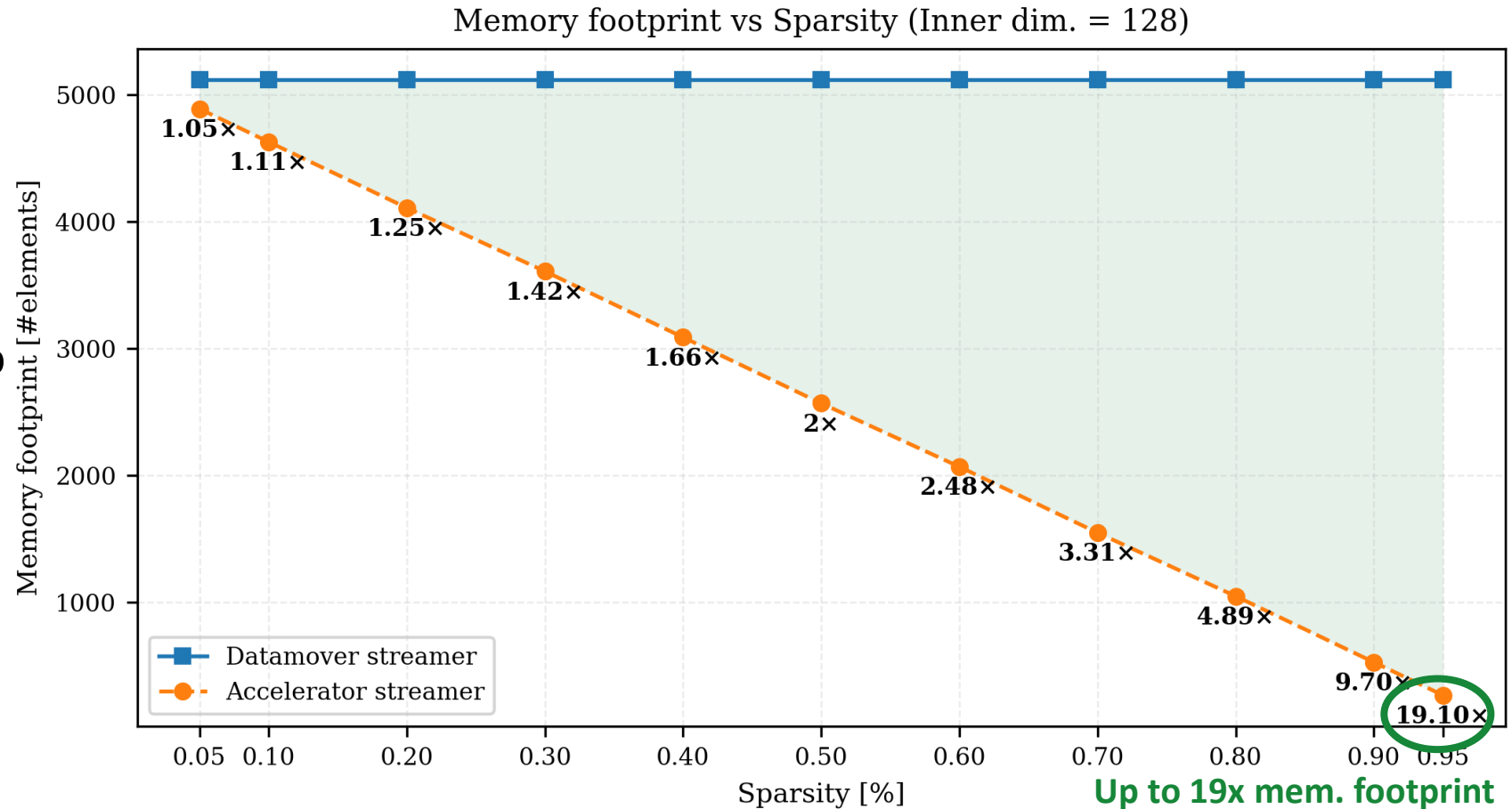


RTL results –Memory footprint VS sparsity



- **Assumptions:**

- MAC units = 4
- X rows = 2
- Y columns = 16
- Inner dim. = 128
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b



Up to 19x mem. footprint
decrease with 95% sparsity



Conclusions

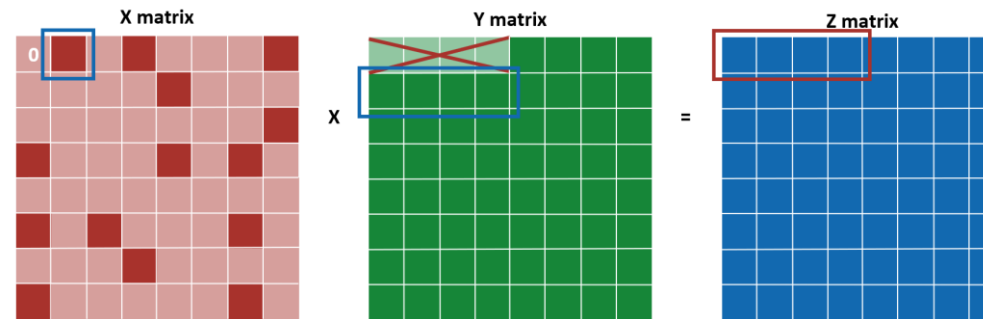
- Taking sparsity into account could reduce latency and memory accesses
- Problem: might be complicated to modify existing systems
- Solution: a flexible and portable accelerator
- From the analytical models Gustavson Z is the best overall
- RTL implementation shows:
 - **2.48x increase in area**
 - **19x decrease in the memory footprint at 95% sparsity!**
 - **15x faster at 95% sparsity!**



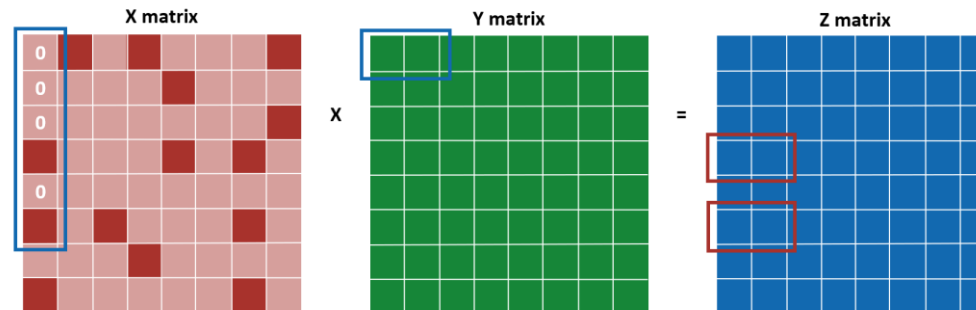
BACKUP Conventions used for the dataflows



- **The engine size will depend on the chosen portions of matrices X and Y**
 - In Gustavson the same X block will be shared with every row of the Y block
 - In outer product the elements of the two blocks are combined to obtain a matrix chunk of partial results



Gustavson's tiling conventions



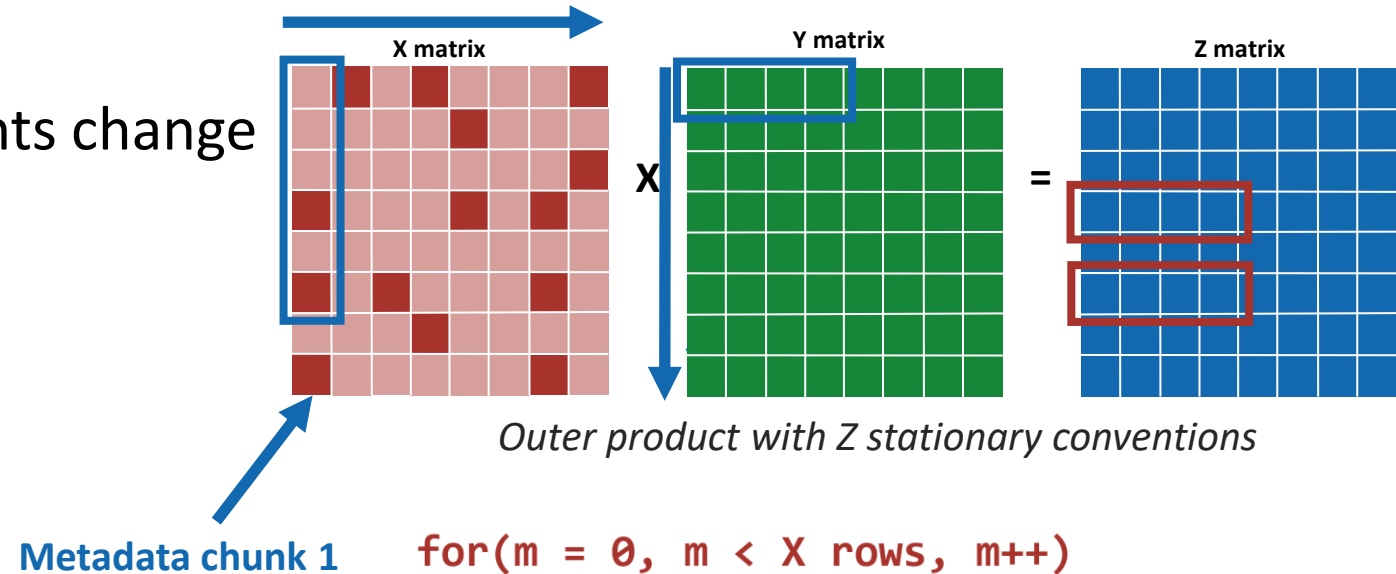
Outer product's tiling conventions



BACKUP Outer product with Z reuse



- Counterintuitively the outer product dataflow with Z reuse performs poorly
 - Lots of metadata loading
 - At each inner iteration Z elements change
 - + mem footprint
 - + latency



```
for(m = 0, m < X rows, m++)  
  for(k = 0, k < Y columns, k++)  
    for(n = 0, n < Inner, n++)
```

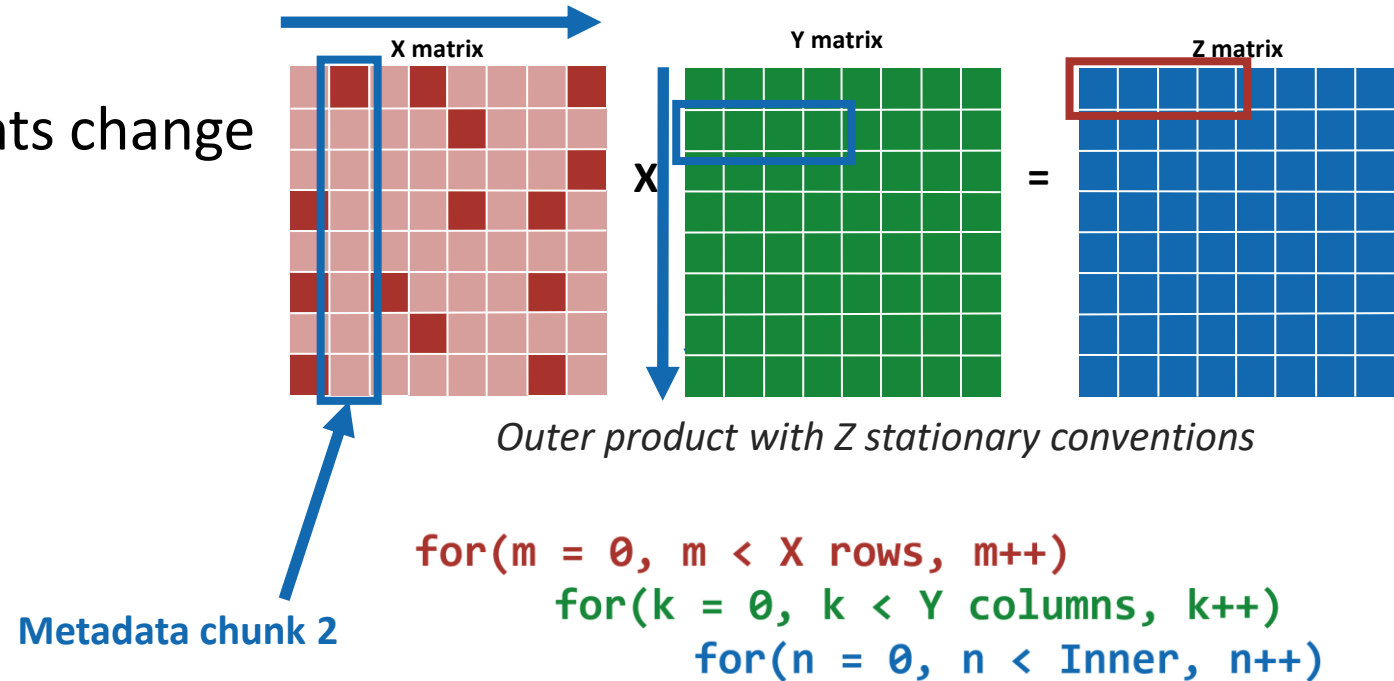
Outer product with Z stationary conventions



BACKUP Outer product with Z reuse



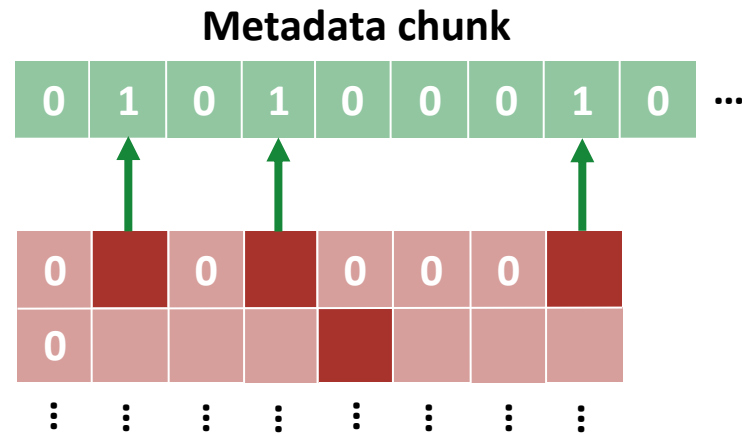
- Counterintuitively the outer product dataflow with Z reuse performs poorly
 - Lots of metadata loading
 - At each inner iteration Z elements change
 - + mem footprint
 - + latency



Outer product with Z stationary conventions



BACKUP –Bitmap format



Row-wise bitmap encoding

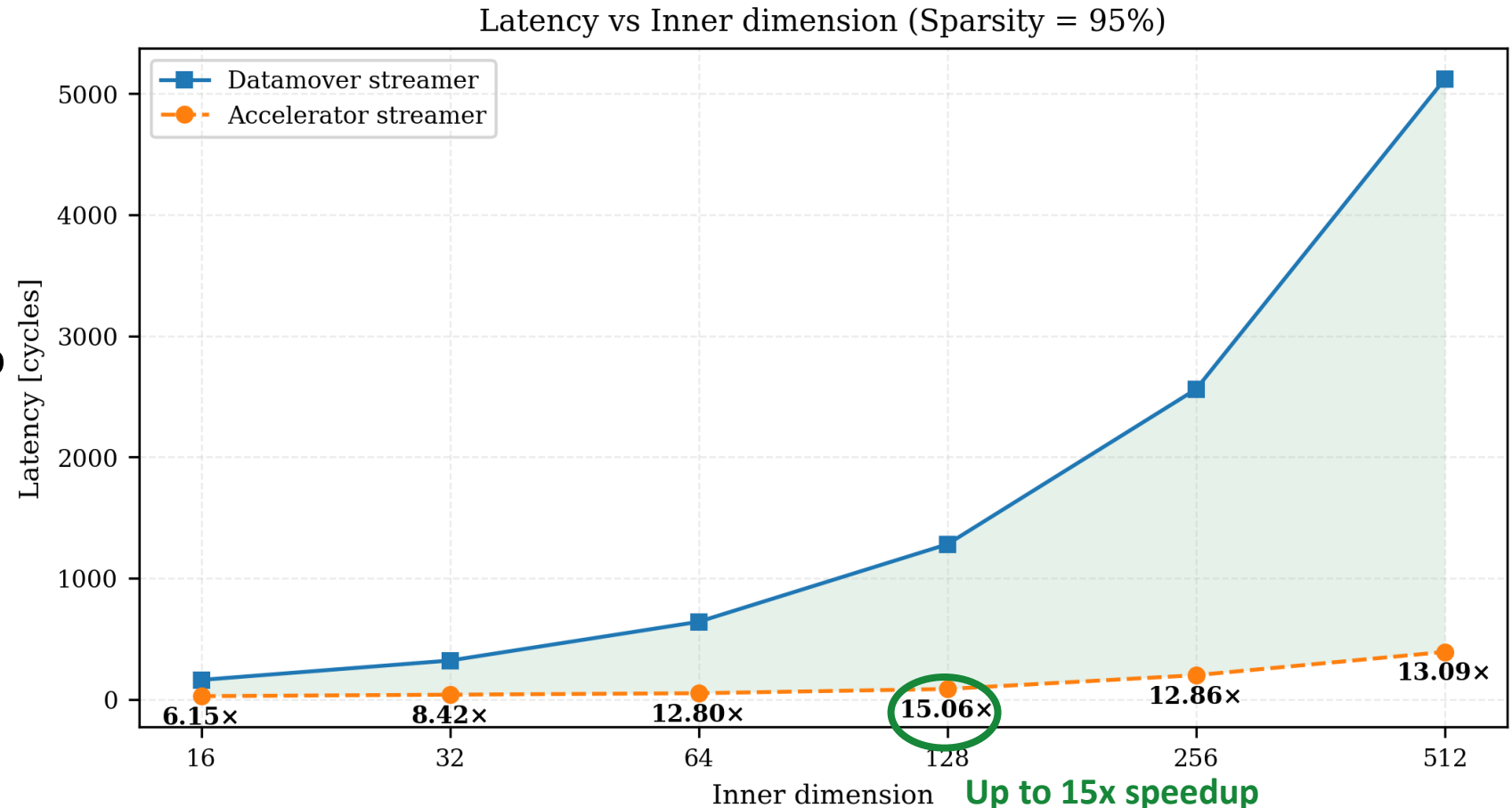


BACKUP RTL results –Latency VS Inner



- **Assumptions:**

- MAC units = 4
- X rows = 2
- Y columns = 16
- Sparsity = 95%
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b



**Up to 15x speedup
with inner dim. of 128**

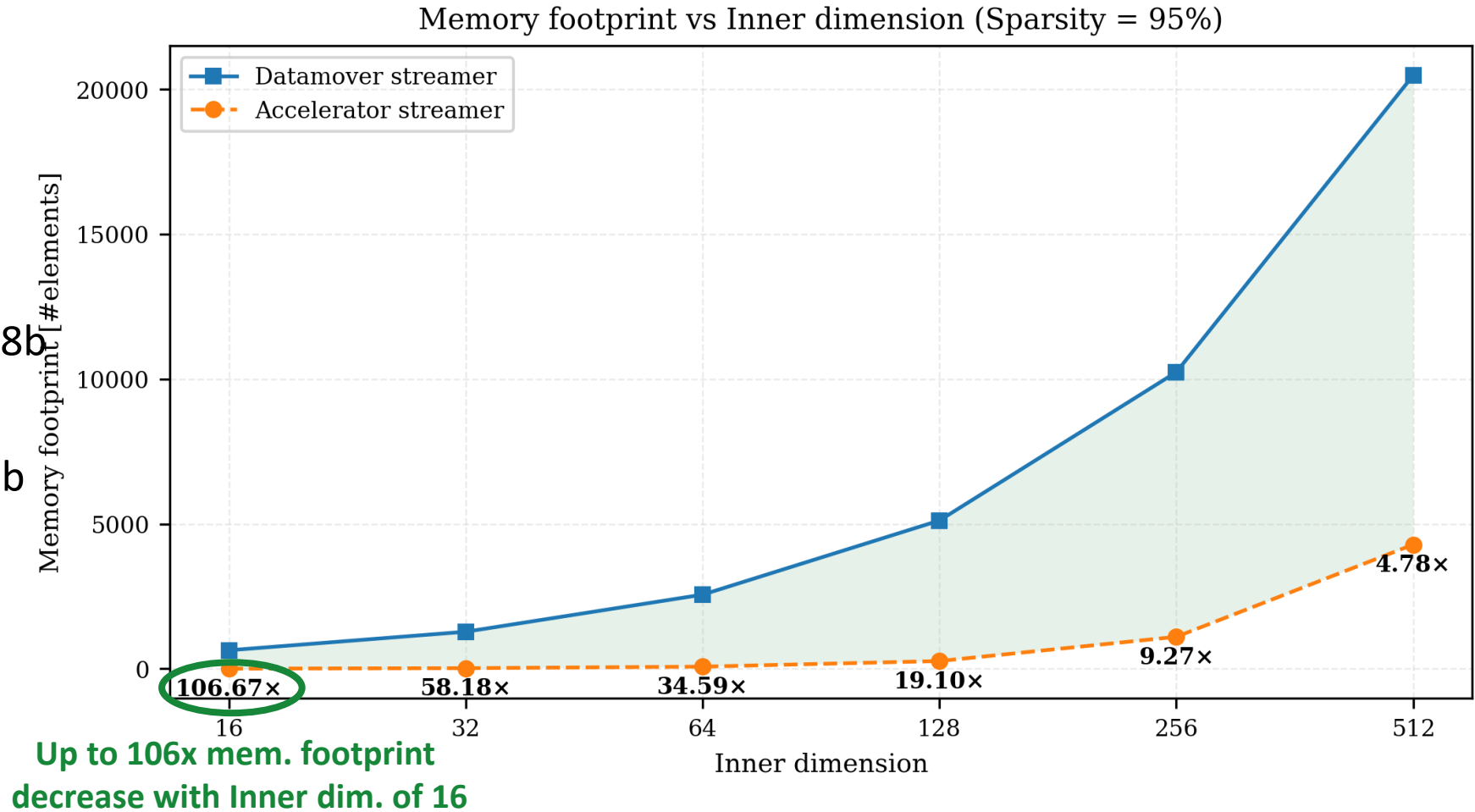


BACKUP RTL results –Memory footprint VS Inner



- **Assumptions:**

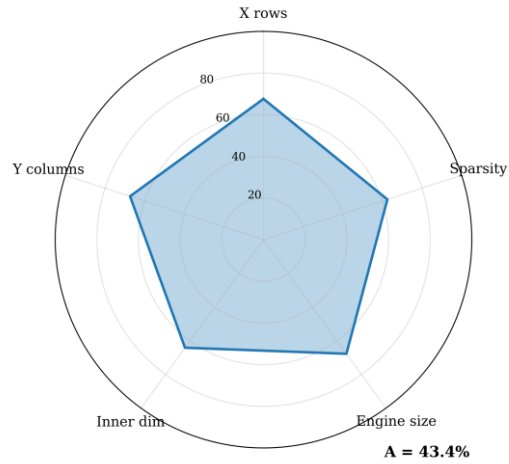
- MAC units = 4
- X rows = 2
- Y columns = 16
- Sparsity = 95%
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b



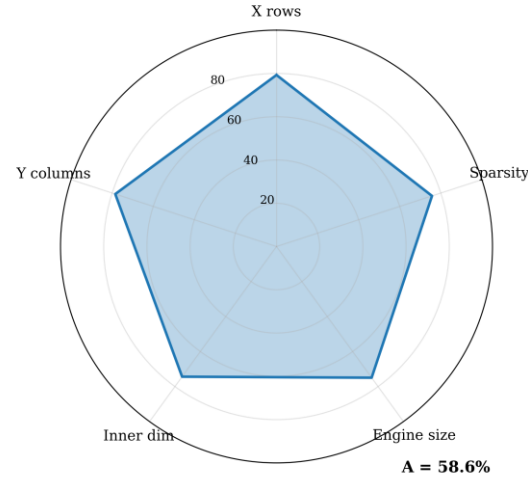
BACKUP Dataflows comparison –BW consumption



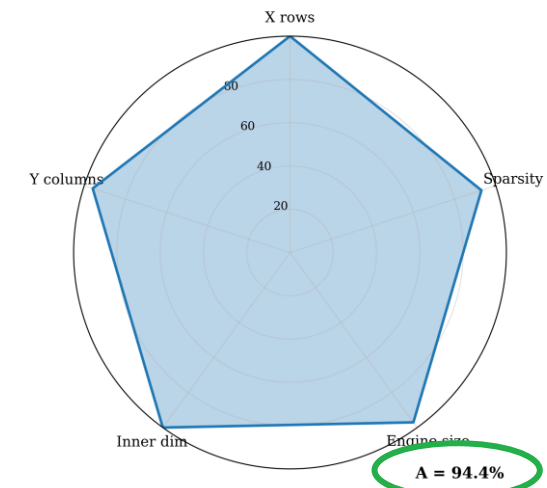
BW consumption - S Gustavson X



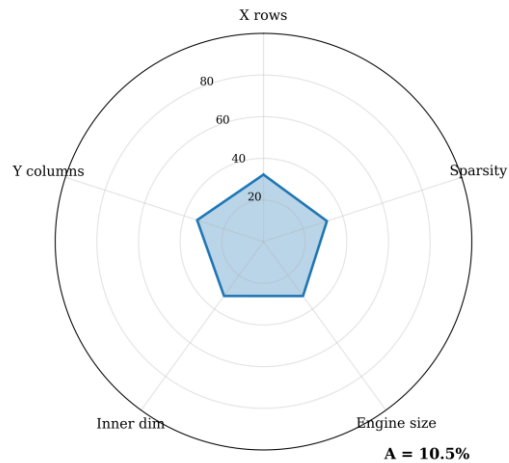
BW consumption - S Gustavson Y



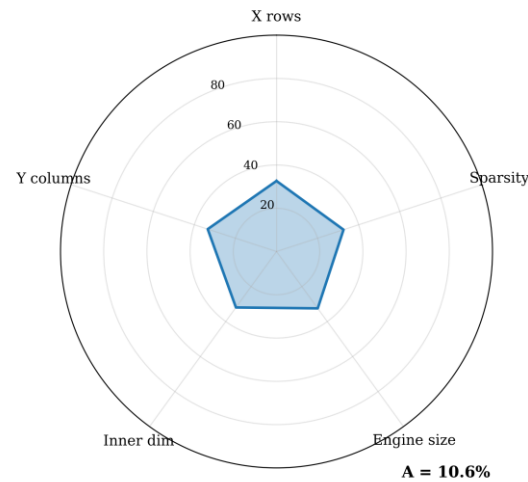
BW consumption - S Gustavson Z



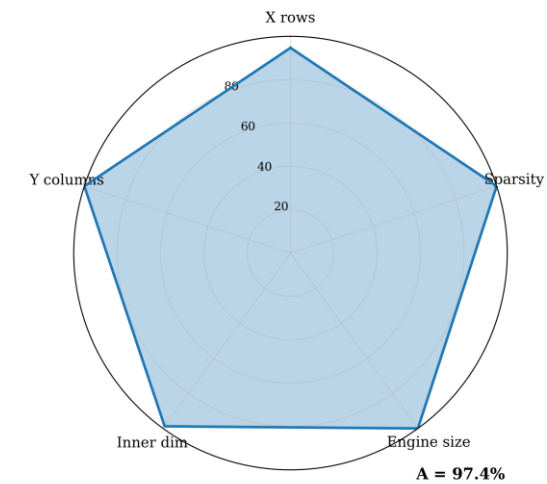
BW consumption - S OuterProd X



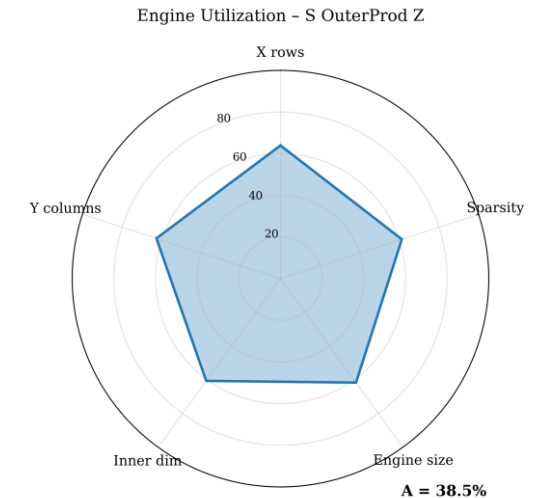
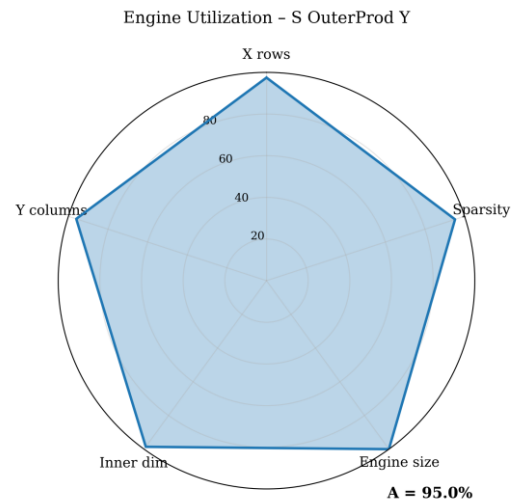
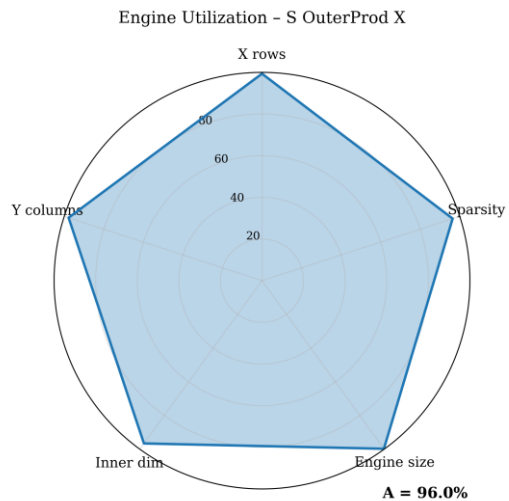
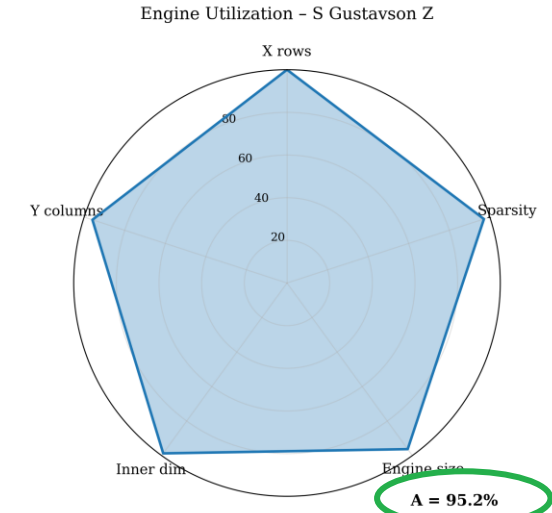
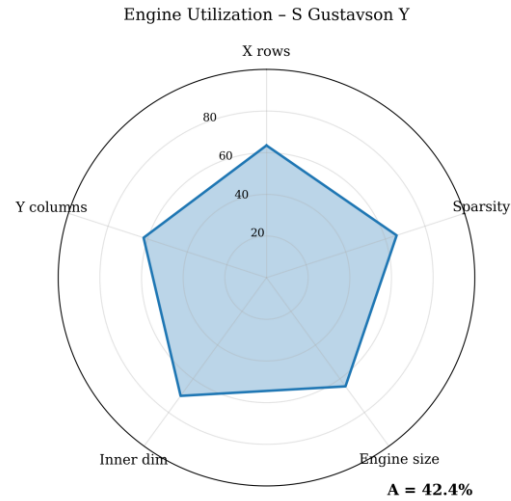
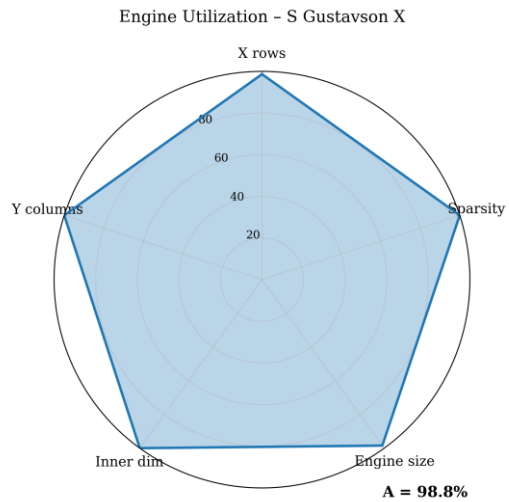
BW consumption - S OuterProd Y



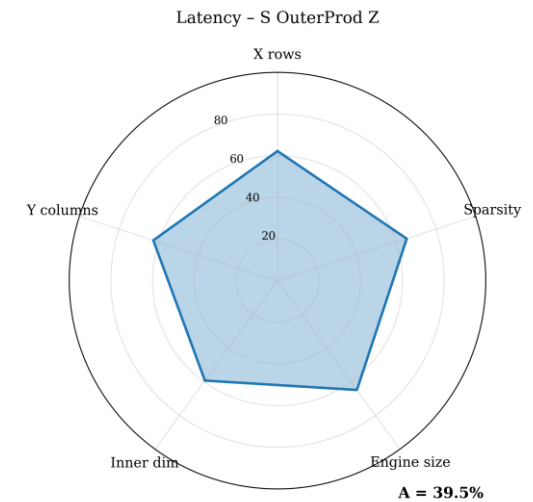
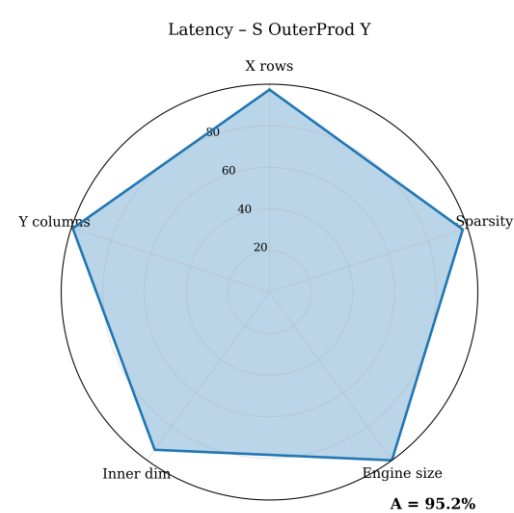
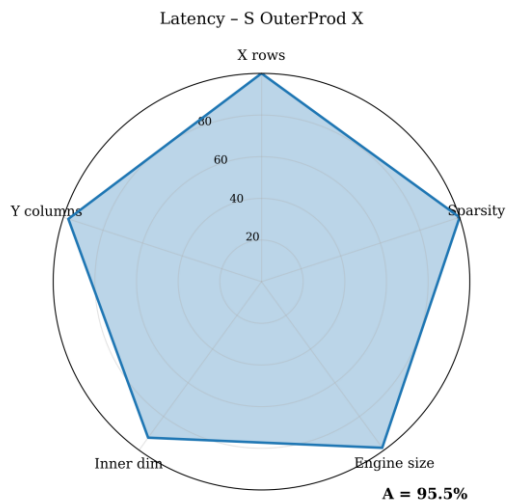
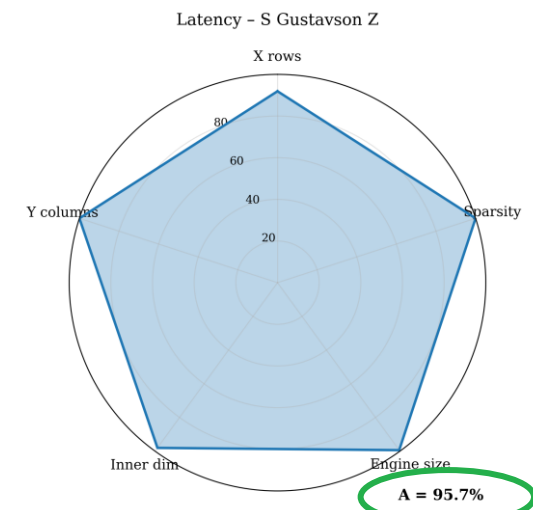
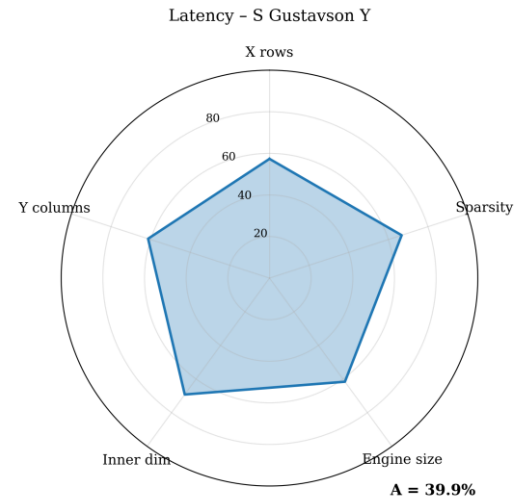
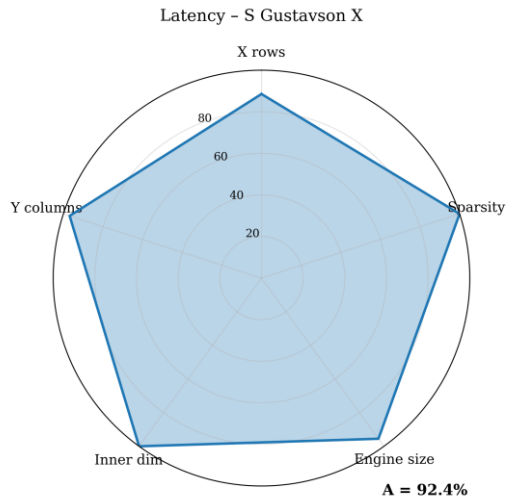
BW consumption - S OuterProd Z



BACKUP Dataflows comparison –Engine utilization



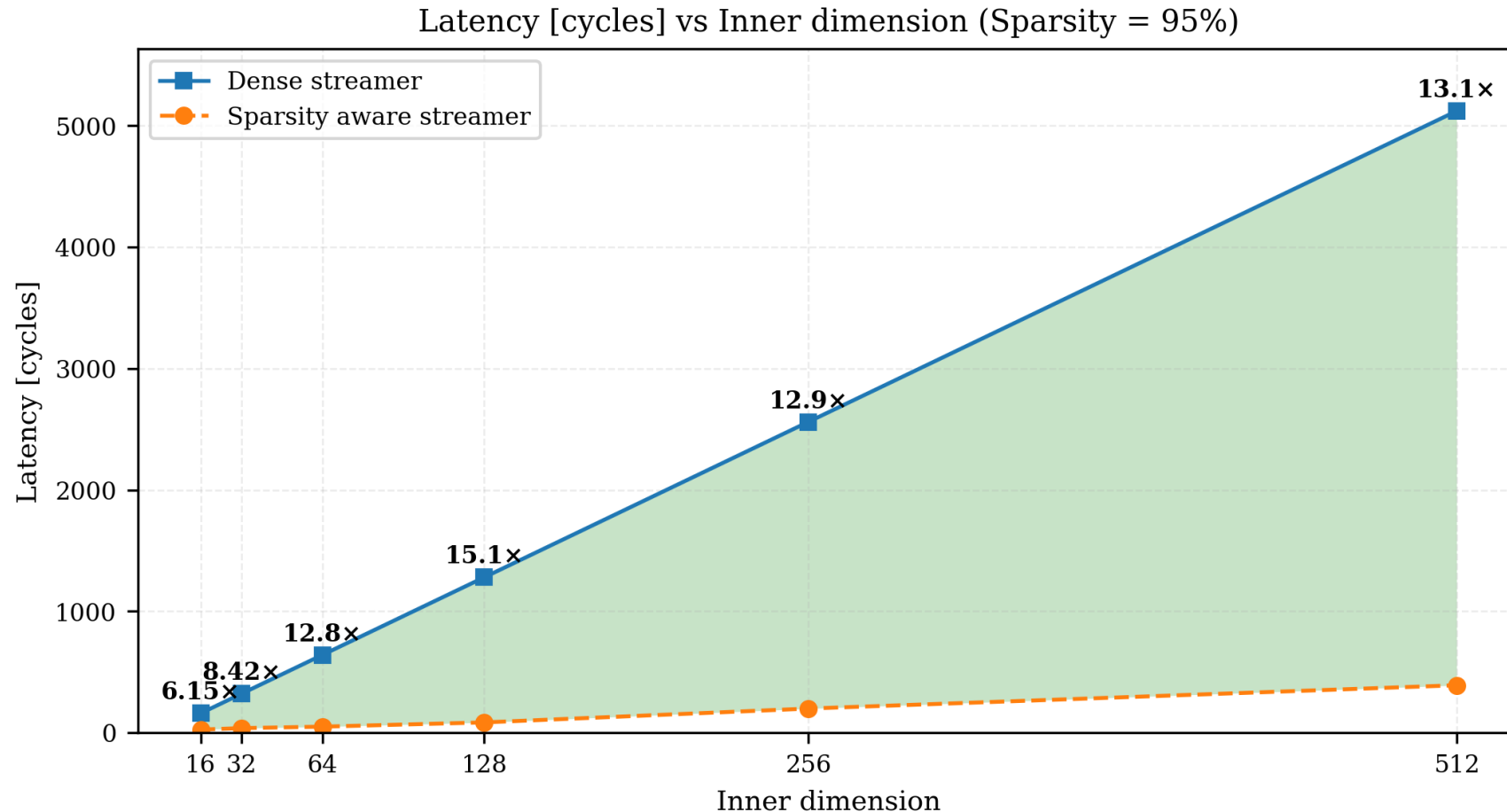
BACKUP Dataflows comparison –Latency



BACKUP RTL results –Inner dimension (latency)



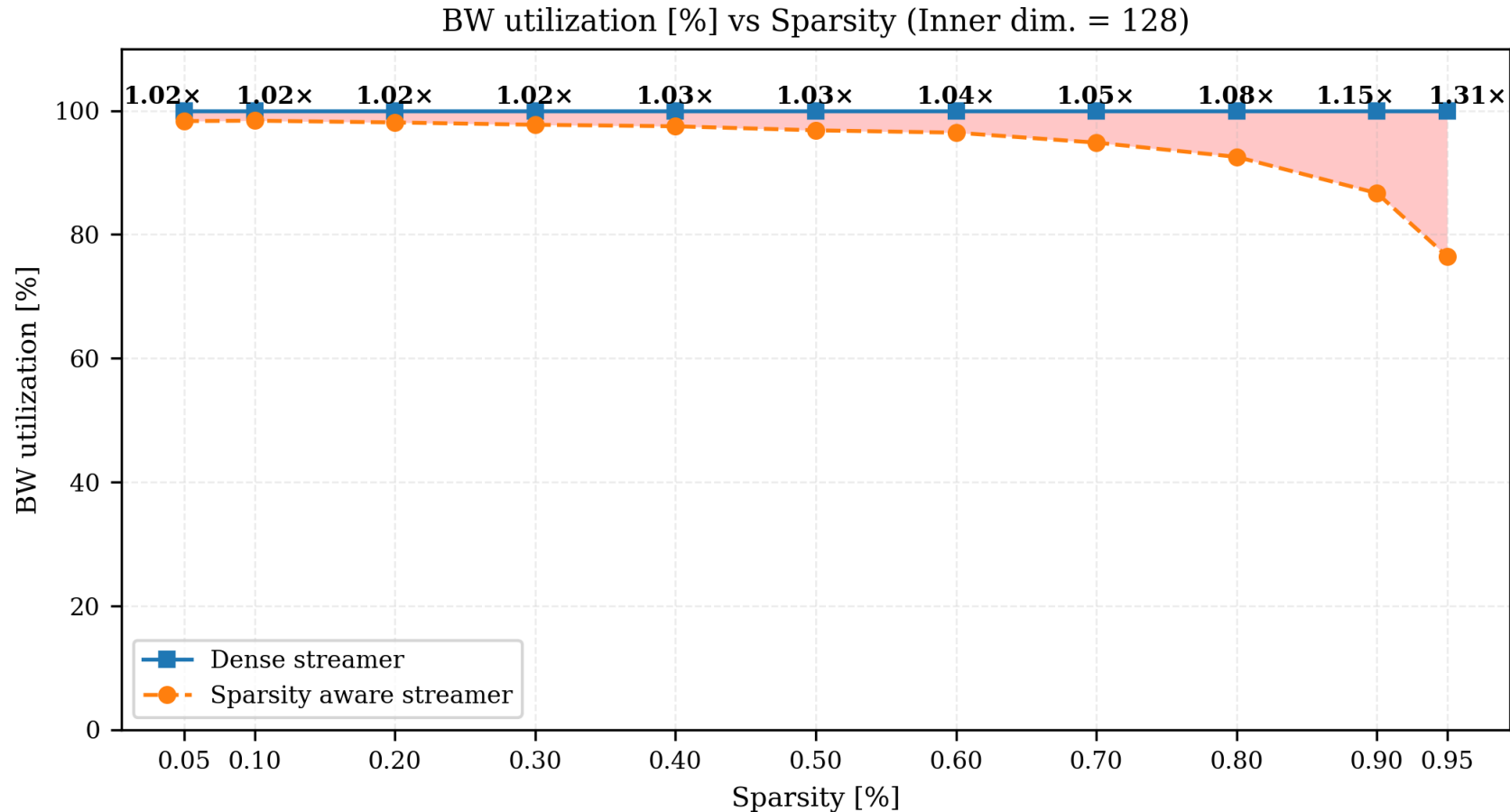
All the results are acquired with an engine size of 4 MAC units, with an X matrix having two rows, a Y matrix having 16 columns and a metadata chunk of 128bits



BACKUP RTL results –Sparsity (BW utilization)



All the results are acquired with an engine size of 4 MAC units, with an X matrix having two rows, a Y matrix having 16 columns and a metadata chunk of 128bits



BACKUP RTL results –Sparsity (engine utilization)



All the results are acquired with an engine size of 4 MAC units, with an X matrix having two rows, a Y matrix having 16 columns and a metadata chunk of 128bits

